VIGILADA MINEDUCACIÓN

UNIVERSIDAD DEL QUINDÍO
Res.MEN 014915 - 02 AGO 2022
RENOVACIÓN ACREDITACIÓN

Asignatura:

Análisis de algoritmos

presentado por:

Melissa Restrepo Hurtado

Alfonso Ocampo Montealegre

Camila Ramirez Galvis

Docente:

Sergio Augusto Cardona Torres

Universidad Del Quindio

2026



UNIQUINDÍO
en conexión territorial
www.uniquindio.edu.co

Tabla de contenido

1. Introducción **2**
1.1 Alcance 3
1.2 Objetivos 4
1.3 Glosario de Términos 6
2. Arquitectura del Sistema 8
2.1 Vista General 8
2.2 Paquetes del Código Fuente 9
2.3 Decisión de Arquitectura: Aplicación Java sin Framework Externo 10
2.4 Diagrama de Clases y Paquetes 10
3. Fuentes de Datos y Preprocesamiento 13
3.1 Extracción de Metadatos 13
3.2 Preprocesamiento de Texto 14
4. Requerimiento 1: Análisis de Similitud Textual 16
4.1 Algoritmos Clásicos 16
4.1.1 Distancia de Levenshtein 16
4.1.2 Needleman-Wunsch 16
4.1.3 Similitud del Coseno sobre TF-IDF 17
4.1.4 Coeficiente de Jaccard 17
4.2 Algoritmos Basados en Inteligencia Artificial 18
4.3 Caso de Estudio 19
4.3.1 Levenshtein: Llenado de la Matriz de Programación Dinámica 20
4.3.2 Needleman-Wunsch: Alineamiento Global de los Tokens 22
4.3.3 Similitud del Coseno sobre TF-IDF: Cálculo de los Pesos y del Ángulo 24
4.3.4 Coeficiente de Jaccard: Conjuntos, Intersección y Unión 26
4.3.5 Los Dos Algoritmos de IA: Embeddings GloVe y FastText 27
4.3.6 Resumen de Resultados del Artículo 1 frente al Artículo 2 30
4.4 Análisis Formal de Complejidad Computacional 35
5. Requerimiento 2: Agrupamiento Jerárquico 36
5.1 Preprocesamiento del Texto 36
5.2 Cálculo de la Similitud: Matriz de Distancias 37
5.3 Aplicación del Clustering: Algoritmo Aglomerativo y Criterios de Enlace 38
5.3.1 Paso a Paso: las Primeras Fusiones 40

5.3.2 Las Últimas Fusiones: Cómo Termina Cada Árbol 41

1. Introducción

La bibliometría estudia la producción científica con métodos cuantitativos: qué se publica, sobre qué temas y cómo se relacionan unos trabajos con otros. Cuando el número de artículos crece, leerlos y compararlos a mano deja de ser viable, y se necesitan algoritmos que midan qué tan parecidos son dos textos, que agrupen automáticamente los trabajos que tratan temas cercanos y que localicen con rapidez un término dentro de un conjunto de documentos. Elegir bien esos algoritmos exige conocer su fundamento matemático y su costo computacional, no solo su resultado.

Este proyecto aplica esa idea a un corpus de 20 artículos científicos sobre Inteligencia Artificial Generativa en Educación. Sobre sus **abstracts** se implementaron desde cero algoritmos de similitud textual, de agrupamiento jerárquico y de búsqueda de patrones, se analizó la complejidad de cada uno y se contrastó ese análisis con lo medido en la práctica. Los resultados se presentan en una aplicación web desplegable.

Este documento describe el diseño, la arquitectura y el análisis algorítmico-matemático de la solución implementada. Está organizado así: la sección 2 presenta la arquitectura; la sección 3, las fuentes de datos y el preprocesamiento; las secciones 4, 5 y 6 desarrollan los Requerimientos 1, 2 y 3; la sección 7 describe la aplicación web; la sección 8, las pruebas de verificación; la sección 9, el despliegue en Azure; y las secciones 10 y 11, las limitaciones y las referencias.

1.1 Alcance

El sistema procesa los **abstracts** de 20 artículos científicos suministrados por el docente, y responde a los requerimientos funcionales de la descripción final del proyecto:

1. **Requerimiento 1:** análisis de similitud textual entre dos o más artículos seleccionados dinámicamente, usando 4 algoritmos clásicos y 2 algoritmos basados en Inteligencia Artificial.
2. **Requerimiento 2:** agrupamiento jerárquico de los 20 abstracts usando 3 criterios de enlace distintos, representado como dendrograma, y evaluado con una métrica de coherencia interna.
3. **Requerimiento 3:** búsqueda de las ocurrencias exactas de un patrón en los 20 abstracts con Knuth-Morris-Pratt, Boyer-Moore y Rabin-Karp, con caso de estudio paso a paso y comparación de la complejidad teórica frente al rendimiento medido.
4. **Requerimiento 4:** despliegue de la aplicación y documentación técnica de cada requerimiento.

La arquitectura se diseñó de forma modular (ver sección 2): el Requerimiento 3, agregado en la descripción final del proyecto, se incorporó como un paquete nuevo (`busqueda`) sin modificar los algoritmos existentes.

Todos los algoritmos de similitud, clustering y búsqueda de patrones fueron **implementados explícitamente con estructuras básicas del lenguaje** (arreglos, mapas, recursión), sin usar funciones de alto nivel de una librería que resuelva directamente el problema, conforme lo exige el enunciado.

Quedan **fuera del alcance**:

- El texto completo de los artículos: el análisis trabaja solo con los abstracts, el título y los autores extraídos de los 20 PDF (sección 3.1).
- La descarga automática de artículos desde bases de datos bibliográficas: el corpus es fijo y lo entregó el docente.
- El entrenamiento de modelos de lenguaje: los dos enfoques de IA usan modelos de *embeddings* ya entrenados (GloVe y FastText), sin ajustarlos al corpus.
- La búsqueda aproximada o semántica de patrones: el Requerimiento 3 encuentra solo coincidencias exactas.
- Textos en otros idiomas: el preprocesamiento (stopwords y stemming) está diseñado para el inglés, el idioma de los 20 abstracts.

1.2 Objetivos

Objetivo general. Implementar, analizar y comparar algoritmos de similitud textual, agrupamiento jerárquico y búsqueda de patrones sobre un corpus de artículos científicos,

fundamentando matemáticamente su funcionamiento y su complejidad computacional, y presentarlos en una aplicación web desplegada.

Objetivos específicos.

1. Implementar 4 algoritmos clásicos de similitud textual (Levenshtein, Needleman-Wunsch, coseno con TF-IDF y Jaccard) y 2 basados en modelos de *embeddings* preentrenados (GloVe y FastText), que comparen dos o más artículos elegidos por el usuario, y explicar paso a paso su cálculo sobre un par de abstracts (Requerimiento 1).
2. Construir desde cero 3 algoritmos de agrupamiento jerárquico aglomerativo (enlace simple, completo y promedio), representar cada resultado como un dendrograma y determinar cuál produce agrupamientos más coherentes con una métrica interna (Requerimiento 2).
3. Implementar los algoritmos de búsqueda de patrones Knuth-Morris-Pratt, Boyer-Moore y Rabin-Karp sin funciones de búsqueda de la librería, mostrar el llenado de sus estructuras auxiliares y contrastar su complejidad teórica con los tiempos medidos de preprocesamiento y de búsqueda (Requerimiento 3).
4. Desplegar la aplicación y documentar técnicamente cada requerimiento (Requerimiento 4).
5. Verificar la corrección de los algoritmos con pruebas que comparen sus resultados contra valores de referencia conocidos o calculados por un método independiente.

1.3 Glosario de Términos

Tabla 1

Glosario de Términos

Término	Definición
Abstract	Resumen de un artículo científico. Es el texto que el sistema analiza de cada uno de los 20 artículos.
Corpus	Conjunto de documentos que se analiza; aquí, los 20 abstracts
Token	Unidad mínima de texto tras dividirlo; en este proyecto, una palabra.
Tokenización	Proceso de dividir un texto en tokens: pasar a minúsculas, quitar signos de puntuación y separar las palabras.
Stopword	Palabra muy frecuente con poco significado propio ("the", "of", "and"), que se elimina antes de comparar textos.
Distancia de edición (Levenshtein)	Número mínimo de inserciones, eliminaciones y sustituciones necesarias para convertir una secuencia en otra.
Alineamiento global (Needleman-Wunsch)	Correspondencia entre dos secuencias completas, que puede incluir huecos, y que maximiza un puntaje de coincidencias y penalizaciones.
TF-IDF	Peso de un término en un documento: su frecuencia en el documento (TF) multiplicada por lo raro que es en el corpus (IDF). Da más peso a los términos que distinguen un documento de los demás.
Similitud del coseno	Coseno del ángulo entre dos vectores: 1 si apuntan en la misma dirección y 0 si no comparten componentes.
Distancia euclidiana	Longitud del segmento que une dos vectores en el espacio.
Coeficiente de Jaccard	Tamaño de la intersección de dos conjuntos dividido entre el tamaño de su unión.

Término	Definición
Embedding (vector de palabra)	Vector de números reales que representa una palabra, aprendido de grandes volúmenes de texto, de modo que palabras de significado parecido quedan cerca.
GloVe / FastText	Modelos de embeddings preentrenados. Se usan GloVe de 100 dimensiones y FastText de 300 dimensiones.
Clustering jerárquico aglomerativo	Agrupamiento que empieza con cada elemento en su propio grupo y fusiona, en cada paso, los dos grupos más cercanos hasta dejar uno solo.
Fórmula de Lance-Williams	Fórmula que actualiza la distancia de un grupo recién fusionado a los demás a partir de las distancias ya conocidas, sin recorrer todos los pares de elementos.
Dendrograma	Árbol que muestra el orden de las fusiones del clustering jerárquico y la distancia (altura) a la que ocurrió cada una.
Distancia cofenética	Altura del dendrograma a la que dos elementos quedan por primera vez en el mismo grupo.
Correlación cofenética	Correlación entre las distancias originales y las distancias cofenéticas; mide qué tan fielmente el dendrograma conserva las distancias reales.
Preprocesamiento	Trabajo que un algoritmo de búsqueda hace una sola vez sobre el patrón (construir sus tablas o su hash) antes de recorrer el texto.
API REST	Conjunto de rutas HTTP que el servidor expone y que la interfaz web consulta para obtener los resultados en formato JSON.
Despliegue	Publicación de la aplicación en un servidor accesible por internet.

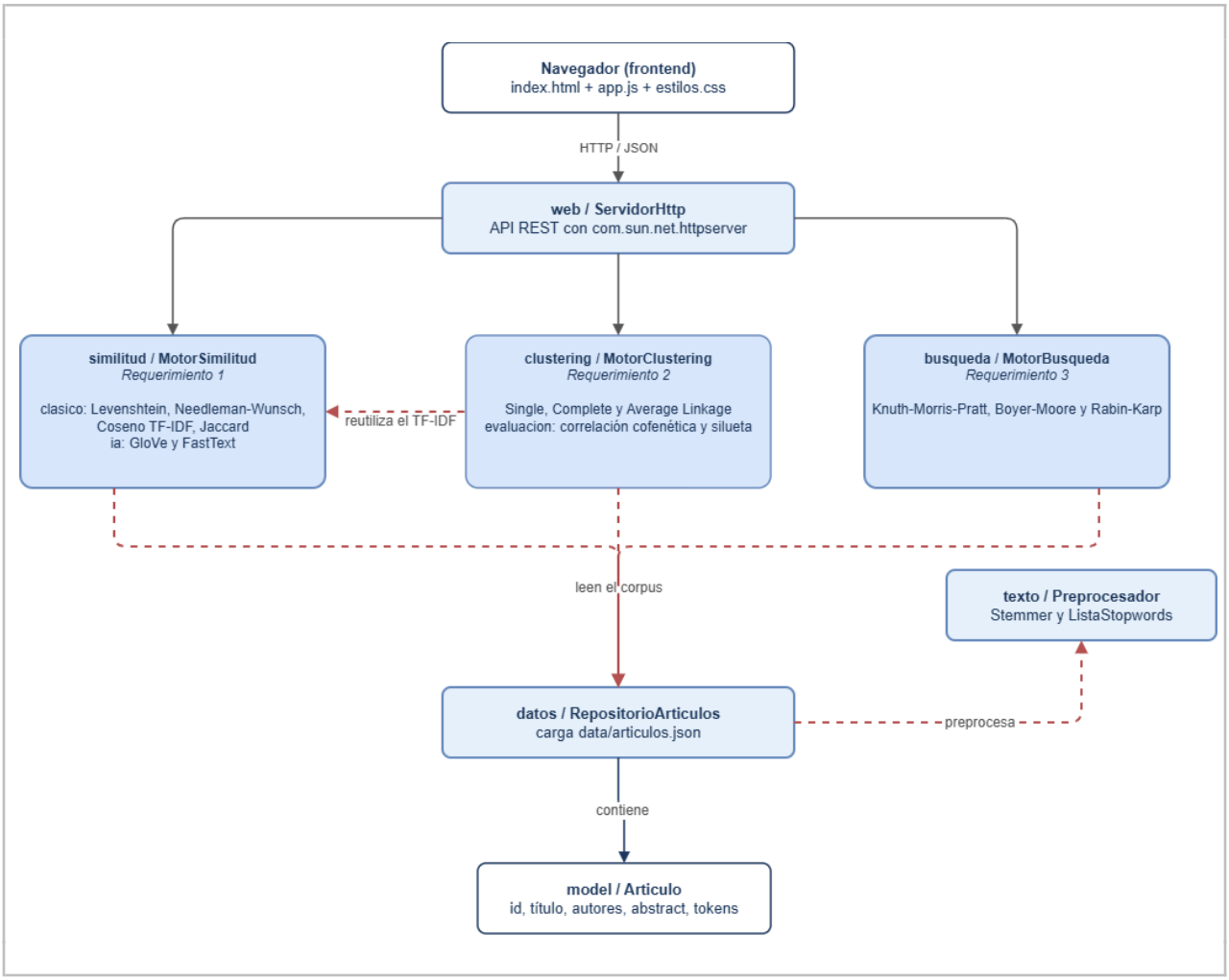
Nota. Elaboración propia.

2. Arquitectura del Sistema

2.1 Vista General

Figura 1

Vista General de la Arquitectura del Sistema



Nota. Elaboración propia.

Cada requerimiento tiene su propio **motor**, que `ServidorHttp` invoca desde una ruta de la API (sección 7). Todos leen los 20 abstracts de `RepositorioArticulos`, que los preprocesa una sola vez al arrancar con el paquete `texto`. La flecha ◄ indica que `MotorClustering` reutiliza el vectorizador TF-IDF ya entrenado por `MotorSimilitud`, para no recalcular el idf del corpus.

2.2 Paquetes del Código Fuente

Tabla 2

Paquetes del Código Fuente y su Responsabilidad

Paquete	Responsabilidad
busqueda	Los 3 algoritmos de búsqueda de patrones (interfaz `AlgoritmoBusqueda`: `KnuthMorrisPratt`, `BoyerMoore`, `RabinKarp`), `ResultadoBusqueda`, `Traza` y `MotorBusqueda` (orquestador, validación y medición de tiempos).
clustering	Algoritmo aglomerativo genérico, las 3 estrategias de enlace, dendrograma.
datos	`RepositorioArticulos`: carga y cachea el corpus desde `data/articulos.json`.
model	Entidad de dominio `Articulo` (id, título, autores, abstract, tokens).
ordenamiento (Seguimiento)	Es el paquete principal donde se encuentra todo lo del Seguimiento por lo tanto se tomará en cuenta en el proyecto.
similitud	`MotorSimilitud` (orquestador) y `ResultadoSimilitud` (DTO de resultado).
texto	`Preprocesador`, `Stemmer`, `ListaStopwords`: pipeline de preprocesamiento de texto.
util	`JsonParser` / `JsonEscritor`: utilidades de lectura/escritura JSON propias.
web	`ServidorHttp`: servidor HTTP, rutas de la API REST, archivos estáticos.

Nota. Elaboración propia.

2.3 Decisión de Arquitectura: Aplicación Java sin Framework Externo

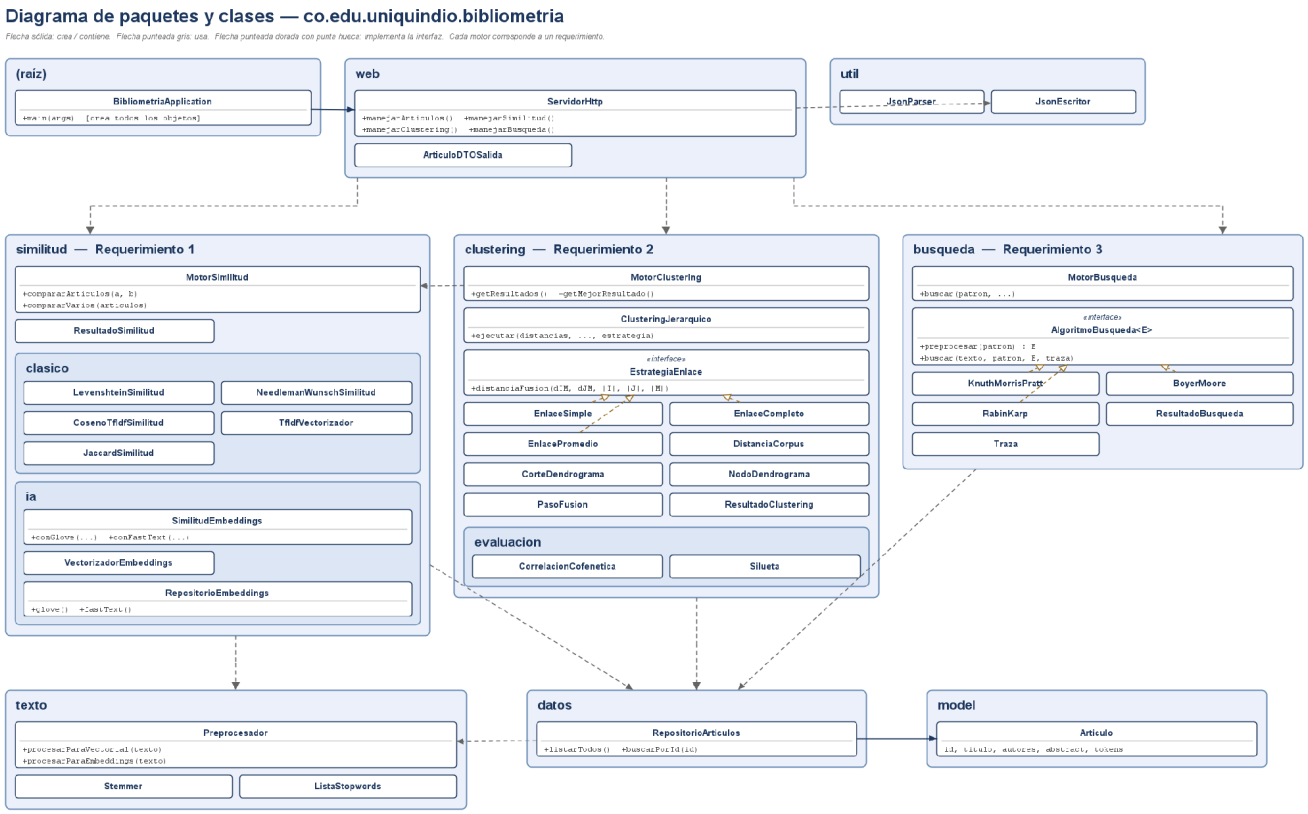
El backend se implementó como una aplicación Java 17 **sin dependencias externas**, usando `com.sun.net.httpserver.HttpServer` (incluido en el JDK) en lugar de Spring Boot, y un parser/escritor JSON propio en lugar de Jackson/Gson. La razón es práctica (el entorno de desarrollo no tenía acceso a Maven Central para verificar la compilación con esas dependencias) pero también tiene una ventaja pedagógica: todo el comportamiento de la aplicación incluyendo la serialización de datos es auditable y no depende de "cajas negras". El `pom.xml` sigue siendo un proyecto Maven estándar y compila igualmente con `mvn clean package` en cualquier entorno con acceso normal a internet. Esta decisión se declara también en la declaración de uso de IA generativa que acompaña a este documento.

2.4 Diagrama de Clases y Paquetes

El siguiente diagrama muestra las 40 clases de la aplicación, agrupadas en sus 11 paquetes, y sus relaciones: creación o contención (línea sólida), dependencia de uso (línea punteada gris) e implementación de una interfaz (línea punteada dorada con punta hueca). `BibliometriaApplication` es el punto de entrada: construye e inyecta manualmente, sin contenedor de inyección de dependencias, todos los objetos del sistema. Ese estilo de *composición explícita* hace visibles y auditables todas las dependencias entre clases. El diagrama se genera con `GenerarDiagramaClases` (carpeta de pruebas).

Figura 2

Diagrama de Clases y Paquetes de la Aplicación



Nota. Elaboración propia.

Cada requerimiento ocupa su propio paquete, con un motor que lo orquesta. Los algoritmos intercambiables se definen detrás de una interfaz (patrón *Strategy*), de modo que el motor los ejecuta todos de la misma forma:

Tabla 3

Paquete, Motor e Interfaz de Cada Requerimiento

Paquete	Requerimiento	Motor	Interfaz / algoritmos	Sección
`similitud (clasico, ia)`	*1*	`MotorSimilitud`	4 clásicos (Levenshtein, Needleman-Wunsch, Coseno TF-IDF, Jaccard) y 2 de IA (`SimilitudEmbeddings` con GloVe y con FastText)	*4*
`clustering (evaluacion)`	*2*	`MotorClustering`	`EstrategiaEnlace`: `EnlaceSimple`, `EnlaceCompleto`, `EnlacePromedio`; métricas `CorrelacionCofenetica` y `Silueta`	*5*
`busqueda`	*3*	`MotorBusqueda`	`AlgoritmoBusqueda<E>`: `KnuthMorrisPratt`, `BoyerMoore`, `RabinKarp`	*6*

Nota. Elaboración propia.

Gracias a esta separación, el Requerimiento 3 y el segundo modelo de IA (FastText) se incorporaron agregando clases nuevas, sin modificar los algoritmos que ya existían.

3. Fuentes de Datos y Preprocesamiento

3.1 Extracción de Metadatos

El corpus está formado por 20 artículos científicos (Alfredo et al., 2024; Bahroun et al., 2023; Bewersdorff et al., 2023; Bond et al., 2024; Chen, Xie y Hwang, 2020; Chen, Xie, Zou et al., 2020; Chiu et al., 2023; Dai y Ke, 2022; Gavhane y Pagare, 2024; Joksimovic et al., 2023; Laupichler et al., 2022; Lo, 2023; Lye y Lim, 2024; Mikeladze et al., 2024; Ogunleye et al., 2024; Ouyang et al., 2023; Tan et al., 2022; Xia et al., 2024; Zawacki-Richter et al., 2019; Zhang y Aslan, 2021), cuyas referencias completas están en la sección 11.

Los 20 PDFs (`recursos/articulos-originales/`) tienen formatos de columnas distintos según la editorial (Elsevier, Springer, MDPI, Emerald, entre otras), lo que hace que la extracción automática de texto con `pdftotext` desordene el abstract en varios casos (el texto de columnas adyacentes se intercala). El proceso seguido fue:

1. Extracción de texto plano de cada PDF (`pdftotext`).
2. Heurística de localización del abstract (entre el encabezado "Abstract"/"A B S T R A C T" y el inicio de la sección "1. Introduction" o la lista de "Keywords").
3. **Revisión manual de los 20 resultados** contra el PDF original, con corrección explícita de los casos donde la heurística fallaba (columnas intercaladas, referencias bibliográficas insertadas a mitad de un párrafo, palabras partidas por guion de fin de línea).
4. Normalización de caracteres especiales (ligaduras tipográficas ﬁ, ﬂ; comillas tipográficas; guiones largos) a su forma ASCII equivalente.

El resultado (`extraction/articulos.json`, copiado como recurso de la aplicación en `app/src/main/resources/data/articulos.json`) es la única fuente de datos que usa la aplicación en tiempo de ejecución: no se re-procesan los PDFs en cada arranque, lo cual hace el comportamiento reproducible y evita tener que empaquetar una librería de extracción de PDF dentro de la aplicación desplegada.

3.2 Preprocesamiento de Texto

Se implementaron **dos variantes** del pipeline de preprocesamiento, elegidas según lo que necesita cada familia de algoritmos:

Tabla 4

Variantes del Preprocesamiento y Algoritmos que las Usan

Pipeline	Pasos	Usado por
`normalizarParaEdicion`	minúsculas, remoción de acentos (NFD), remoción de puntuación	Levenshtein
`procesarParaVectorial`	tokenizar = quitar stopwords, **stemming** (Porter simplificado)	Needleman-Wunsch (a nivel de tokens), TF-IDF/Coseno, Jaccard
`procesarParaEmbeddings`	tokenizar = quitar stopwords (**sin** stemming)	Los 2 algoritmos de IA (embeddings GloVe y FastText)

Nota. Elaboración propia.

Nota de diseño importante: los algoritmos basados en embeddings usan tokens **sin stemming**, a diferencia de TF-IDF/Jaccard. La razón es que los modelos de embeddings preentrenados (GloVe, FastText) tienen vectores para palabras reales del idioma inglés ("literacy", "assessment"), pero no para raíces truncadas artificiales que produce el stemmer ("literaci", "assess"). Aplicar stemming antes de la búsqueda en el vocabulario de embeddings

reduciría drásticamente la cantidad de palabras que logran encontrar su vector (en pruebas tempranas del proyecto, la cobertura pasó de ~46% a **97%–100%** de las palabras del abstract al corregir este detalle). Este es exactamente el tipo de matiz algorítmico que el proyecto pide analizar críticamente.

El **stemmer** (`texto.Stemmer`) es una implementación propia y simplificada del algoritmo de Porter (1980): calcula la "medida" silábica (m) de una palabra a partir de su patrón de vocales/consonantes, y aplica reglas de sufijos en 5 pasos sucesivos (por ejemplo, `"sses"`→`"ss"`, `"eed"`→`"ee"` si m>0, `"ational"`→`"ate"` si m>0, etc.). No se usó ninguna librería de stemming (Snowball, Lucene Analyzer, etc.).

Las **stopwords** (`texto.ListaStopwords`) son una lista estática de ~180 palabras vacías del inglés (idioma de los 20 abstracts), definida manualmente.

4. Requerimiento 1: Análisis de Similitud Textual

4.1 Algoritmos Clásicos

4.1.1 Distancia de Levenshtein

Propuesta por Levenshtein (1966), mide el número mínimo de inserciones, borrados y sustituciones de un carácter para transformar la cadena A en la cadena B, mediante programación dinámica en el código lo podemos ver en LevenshteinSimilitud.java método construirMatriz:

```
D[0][j] = j                                          (Insertar j caracteres)
D[i][0] = i                                          (Borrar i caracteres)
D[i][j] = D[i-1][j-1]                                Si A[i] == B[j]
D[i][j] = 1 + min(D[i-1][j], D[i][j-1], D[i-1][j-1]) en otro caso
```

$$\text{Similitud} = 1 - \frac{\text{distancia}}{\max(|A|, |B|)}$$

El resultado se normaliza como `similitud`, se aplica sobre el texto completo normalizado (nivel de carácter), **Implementación:** `similitud.clasico.LevenshteinSimilitud`.

4.1.2 Needleman-Wunsch

Algoritmo clásico de alineamiento global de secuencias (Needleman y Wunsch, 1970). A diferencia de Levenshtein, aquí se aplica **a nivel de tokens/palabras** ya preprocesadas (tras tokenizar, quitar stopwords y aplicar stemming), con un esquema de puntuación `match = +2`, `mismatch = -1`, `gap = -1`:

```
F[0][j] = j · gap
F[i][0] = i · gap
F[i][j] = max( F[i-1][j-1] + s(Ai, Bj),  F[i-1][j] + gap,  F[i][j-1] + gap )
```

Esta elección metodológica (carácter vs. token) es intencional: permite comparar en el análisis crítico (sección 4.4) cómo la granularidad del algoritmo de alineamiento afecta el resultado.

Implementación: `similitud.clasico.NeedlemanWunschSimilitud`.

4.1.3 Similitud del Coseno sobre TF-IDF

Cada abstract se representa como un vector disperso donde cada dimensión es un término del corpus, ponderado por TF-IDF (Salton y McGill, 1983):

$$tf(t,d) = \frac{\text{ocurrencias}(t,d)}{|d|}$$

$$idf(t) = \ln\left(\frac{(1+N)}{(1+df(t))}\right) + 1 \quad (N = 20 \text{ documentos, } df = \text{documentos que contienen } t)$$

$$tfidf(t,d) = tf(t,d) \cdot idf(t)$$

$$\cos(\theta) = \frac{(A \cdot B)}{(||A|| \cdot ||B||)}$$

El IDF se calcula **una sola vez sobre los 20 abstracts completos** (no solo sobre el par seleccionado), para que los pesos reflejen qué tan específico es un término en todo el corpus.

Implementación: `similitud.clasico.TfIdfVectorizador` + `CosenoTfIdfSimilitud`.

4.1.4 Coeficiente de Jaccard

El coeficiente de Jaccard compara conjuntos. Cada abstract se representa como el **conjunto** de tokens distintos que contiene:

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

La intersección y la unión se calculan explícitamente, sin las operaciones de conjuntos de la librería (`retainAll`, `addAll`):

1. Se recorre cada lista de tokens una vez y se conserva la primera aparición de cada término. La tabla hash se usa solo para consultar si un término ya está.
2. **Intersección:** se recorre el conjunto más pequeño y se cuenta cada término que también pertenece al otro.
3. **Unión** por inclusión-exclusión: $|A \cup B| = |A| + |B| - |A \cap B|$. Las palabras comunes se contarían dos veces al sumar $|A| + |B|$, así que se restan una vez.

Implementación: `similitud.clasico.JaccardSimilitud`.

4.2 Algoritmos Basados en Inteligencia Artificial

Los modelos preentrenados Word2Vec, FastText o modelos de lenguaje, calculan la similitud métrica (distancia euclidiana o coseno) entre los vectores. Por eso los dos algoritmos de IA se diferencian en **cómo se generan los embeddings**: cada uno usa un modelo preentrenado distinto, con un método de entrenamiento distinto.

Preparación de los modelos. Los modelos completos pesan cientos de MB. Cada uno se descargó una sola vez y se **filtró** para conservar solo las palabras que aparecen en los 20 abstracts: GloVe con `extraction/build_embeddings.py` (`embeddings/glove100_filtrado.tsv`, 0.9 MB) y FastText con `FiltrarEmbeddingsFastText` (carpeta de pruebas), que lee directamente el archivo `wiki-news-300d-1M.vec.zip` (682 MB) y usa el mismo preprocesamiento de la aplicación (→ `embeddings/fasttext300_filtrado.tsv`, 2.2 MB). Este filtrado es infraestructura

de preparación de datos, no uno de los algoritmos exigidos. FastText distingue mayúsculas; si una palabra solo aparece con mayúscula (por ejemplo la sigla "AIED") se usa esa variante.

Vector de cada abstract. En ambos enfoques el abstract se representa como el **promedio de los vectores de palabra** ("average word embeddings") de sus tokens sin stemming (sección 3.2). Las palabras sin vector en el modelo se omiten:

$$V(\text{documento}) = \frac{1}{K} \cdot \sum \text{embedding}(\text{Palabra_i})$$

Para las K palabras del documento que tienen vector en el modelo

Similitud métrica. Sobre los dos vectores promedio se calcula, en un solo recorrido de las d componentes (d = 100 o 300):

$$\cos(\theta) = \frac{(V_A \cdot V_B)}{(||V_A|| \cdot ||V_B||)}$$ → Similitud que reporta el algoritmo, en [−1, 1]

$$\text{dist}(A,B) = \text{sqrt}(\sum (V_A,i - V_B,i)^2)$$ → Distancia euclidiana, incluida en el detalle del resultado

4.3 Caso de Estudio

Se seleccionaron dos abstracts reales del corpus:

- **Artículo 1 (A):** *"Artificial intelligence literacy in higher and adult education: A scoping literature review"* (Laupichler et al., 2022). Tiene 1129 caracteres tras la normalización, 97 tokens con stemming (76 términos distintos) y 97 tokens sin stemming.
- **Artículo 2 (B):** *"Systematic literature review on opportunities, challenges, and future research recommendations of artificial intelligence in education"* (Chiu et al., 2023).

Tiene 887 caracteres, 70 tokens con stemming (52 términos distintos) y 70 tokens sin stemming.

A continuación se demuestra, con **estos dos abstracts**, cómo calcula cada algoritmo su resultado. Todos los números fueron producidos por el código real de cada algoritmo (`app/src/test/java/.../GenerarCasoSimilitud.java`), al finalizar el cálculo de cada algoritmo podrá visualizar un video donde se realizan los cálculos a mano para mayor entendimiento sobre su funcionamiento. En la interfaz, la pestaña "Requerimiento 1" con los artículos 1 y 2 seleccionados muestra los mismos valores.

4.3.1 Levenshtein: Llenado de la Matriz de Programación Dinámica

Se compara el texto normalizado (minúsculas, sin acentos ni puntuación) carácter por carácter:

```
A: "since artificial intelligence ai is finding its way into mor ..."      (n = 1129)
B: "applications of artificial intelligence in education aied ar ..."      (m = 887)
La matriz D tiene (1129 + 1) × (887 + 1) = 1003440 celdas
D[i][j] es la distancia entre los primeros i caracteres de A y los primeros
j de B
D[i][0] = i,  D[0][j] = j
D[i][j] = D[i-1][j-1]                                   Si A[i] = B[j]
D[i][j] = 1 + min(D[i-1][j-1], D[i-1][j], D[i][j-1])    Si A[i] ≠ B[j]
```

Esquina superior izquierda (D[0..10][0..10]; _ es el espacio):

Tabla 5

Esquina Superior Izquierda de la Matriz de Levenshtein

i/j	0 ø	1 a	2 p	3 p	4 l	5 i	6 c	7 a	8 t	9 i	10 o
0 ø	0	1	2	3	4	5	6	7	8	9	10
1 s	1	1	2	3	4	5	6	7	8	9	10
2 i	2	2	2	3	4	4	5	6	7	8	9
3 n	3	3	3	3	4	5	5	6	7	8	9
4 c	4	4	4	4	4	5	5	6	7	8	9
5 e	5	5	5	5	5	5	6	6	7	8	9
6 _	6	6	6	6	6	6	6	7	7	8	9
7 a	7	6	7	7	7	7	7	6	7	8	9
8 r	8	7	7	8	8	8	8	7	7	8	9
9 t	9	8	8	8	9	9	9	8	7	8	9
10 i	10	9	9	9	9	9	10	9	8	7	8

Nota. Elaboración propia.

Para ver la construcción de algunas celdas lo invitamos a visualizar el siguiente video recordando cambiar la calidad a 1080p: Video 1 Algoritmo de Levenshtein

Tabla 6

Esquina Inferior Derecha de la Matriz de Levenshtein

i/j	877 e	878 d	879 _	880 r	881 e	882 s	883 e	884 a	885 r	886 c	887 h
1119 t	762	761	760	759	759	759	759	759	760	761	762
1120 h	763	762	761	760	760	760	760	760	760	761	761
1121 e	763	763	762	761	760	761	760	761	761	761	762
1122 _	764	764	763	762	761	761	761	761	762	762	762
1123 r	765	765	764	763	762	762	762	762	761	762	763
1124 e	766	766	765	764	763	763	762	763	762	762	763
1125 s	767	767	766	765	764	763	763	763	763	763	763
1126 u	768	768	767	766	765	764	764	764	764	764	764
1127 l	769	769	768	767	766	765	765	765	765	765	765

i/j	877 e	878 d	879 _	880 r	881 e	882 s	883 e	884 a	885 r	886 c	887 h
1128 t	770	770	769	768	767	766	766	766	766	766	766
1129 s	771	771	770	769	768	767	767	767	767	767	767

Nota. Elaboración propia.

Para ver la construcción de algunas celdas y el cálculo de la similitud lo invitamos a ver el siguiente video recordando cambiar la calidad a 1080p: Video 2 Algoritmo Levenshtein

4.3.2 Needleman-Wunsch: Alineamiento Global de los Tokens

Se alinean las secuencias de tokens preprocesados (sin stopwords, con stemming), con `match = +2`, `mismatch = −1` y `gap = −1`:

```
A (97 tokens): sinc, artifici, intellig, ai, find, wai, area,
    everydai, life, improv, ai, skill, ...

B (70 tokens): applic, artifici, intellig, educ, ai, emerg, new,
    research, practition, alik, review, relev, ...

F[i][0] = -i ,   F[0][j] = -j

F[i][j] = max( F[i-1][j-1] + s(A[i], B[j]),
               F[i-1][j] - 1 ,
               F[i][j-1] - 1 )

s = +2  si iguales ,   -1  si no
```

Esquina superior izquierda (F[0..8][0..8]):

Tabla 7

Esquina Superior Izquierda de la Matriz de Needleman-Wunsch

i/j	ø	1 applic	2 artifici	3 intellig	4 educ	5 ai	6 emerg	7 new	8 research
ø	0	-1	-2	-3	-4	-5	-6	-7	-8
1 sinc	-1	-1	-2	-3	-4	-5	-6	-7	-8
2 artifici	-2	-2	1	0	-1	-2	-3	-4	-5
3 intellig	-3	-3	0	3	2	1	0	-1	-2
4 ai	-4	-4	-1	2	2	4	3	2	1
5 find	-5	-5	-2	1	1	3	3	2	1
6 wai	-6	-6	-3	0	0	2	2	2	1
7 area	-7	-7	-4	-1	-1	1	1	1	1
8 everydai	-8	-8	-5	-2	-2	0	0	0	0

Nota. Elaboración propia.

Para ver la construcción de algunas celdas lo invitamos a ver el siguiente video recordando cambiar la calidad del a 1080p: Video 3 Algoritmo de Needleman Wunsch

Tabla 8

Esquina Inferior Derecha de la Matriz de Needleman-Wunsch

i/j	64 major	65 challeng	66 provid	67 suggest	68 futur	69 direct	70 ai
91 expert	-55	-56	-57	-58	-59	-60	-61
92 lastli	-56	-56	-57	-58	-59	-60	-61
93 sever	-57	-57	-57	-58	-59	-60	-61
94 recommend	-58	-58	-58	-58	-59	-60	-61
95 practic	-59	-59	-59	-59	-59	-60	-61
96 deriv	-60	-60	-60	-60	-60	-60	-61
97 result	-61	-61	-61	-61	-61	-61	-61

Nota. Elaboración propia.

El puntaje final es **F[97][70] = −61**. Reconstruyendo el camino óptimo hacia atrás, el alineamiento tiene 13 coincidencias, 54 sustituciones y 33 gaps: 13·(+2) + 54·(−1) + 33·(−1) = 26 − 54 − 33 = −61. Normalización con L = max(97, 70) = 97 (máximo posible 2L = 194; mínimo −L = −97, todo sustituciones o gaps):

$$\text{similitud} = \frac{(-61 - (-97))}{(194 - (-97))} = \frac{36}{291} = 0.1237$$

Para ver la construcción de las últimas celdas y posterior calcular la similitud lo invitamos a visualizar el siguiente video recordando cambiar la calidad a 1080p: Video 4 Algoritmo Needleman Wunsch

4.3.3 Similitud del Coseno sobre TF-IDF: Cálculo de los Pesos y del Ángulo

Con N = 20 documentos, para cada término t de cada abstract d:

$$tf(t,d) = \frac{\text{ocurrencias de t en d}}{\text{tokens de d}}$$

$$idf(t) = \ln\left(\frac{(1+N)}{(1+df(t))}\right) + 1 \quad (df = \text{abstracts del corpus que contienen t})$$

$$w(t,d) = tf(t,d) \cdot idf(t)$$

El vector de A tiene 76 dimensiones (términos distintos) y el de B tiene 52. Solo los **15 términos que comparten** aportan al producto punto. Este es su cálculo completo:

Tabla 9

Cálculo de los Pesos TF-IDF de los Términos Comunes

Término	Cuenta A	tf A	Cuenta B	tf B	df	idf	w A	w B	w A · w B
ai	7	7/97 = 0.0722	7	7/70 = 0.1000	19	ln(21/20)+1 = 1.0488	0,0757	0,1049	0,007938
educ	3	3/97 = 0.0309	3	3/70 = 0.0429	18	ln(21/19)+1 = 1.1001	0,0340	0,0471	0,001604
literatur	2	2/97 = 0.0206	2	2/70 = 0.0286	11	ln(21/12)+1 = 1.5596	0,0322	0,0446	0,001433
content	2	2/97 = 0.0206	1	1/70 = 0.0143	6	ln(21/7)+1 = 2.0986	0,0433	0,0300	0,001297
result	2	2/97 = 0.0206	1	1/70 = 0.0143	8	ln(21/9)+1 = 1.8473	0,0381	0,0264	0,001005
technologi	1	1/97 = 0.0103	3	3/70 = 0.0429	12	ln(21/13)+1 = 1.4796	0,0153	0,0634	0,000967
learn	1	1/97 = 0.0103	3	3/70 = 0.0429	12	ln(21/13)+1 = 1.4796	0,0153	0,0634	0,000967
identifi	2	2/97 = 0.0206	1	1/70 = 0.0143	10	ln(21/11)+1 = 1.6466	0,0340	0,0235	0,000799
relev	1	1/97 = 0.0103	1	1/70 = 0.0143	7	ln(21/8)+1 = 1.9651	0,0203	0,0281	0,000569
current	1	1/97 = 0.0103	1	1/70 = 0.0143	9	ln(21/10)+1 = 1.7419	0,0180	0,0249	0,000447
understand	1	1/97 = 0.0103	1	1/70 = 0.0143	10	ln(21/11)+1 = 1.6466	0,0170	0,0235	0,000399
futur	1	1/97 = 0.0103	1	1/70 = 0.0143	11	ln(21/12)+1 = 1.5596	0,0161	0,0223	0,000358
us	1	1/97 = 0.0103	1	1/70 = 0.0143	15	ln(21/16)+1 = 1.2719	0,0131	0,0182	0,000238
intellig	1	1/97 = 0.0103	1	1/70 = 0.0143	20	ln(21/21)+1 = 1.0000	0,0103	0,0143	0,000147
artifici	1	1/97 = 0.0103	1	1/70 = 0.0143	20	ln(21/21)+1 = 1.0000	0,0103	0,0143	0,000147
Suma (producto punto)									**0,018316**

Nota. Elaboración propia.

Para ver la construcción de algunas filas donde se calcularon los pesos y el angulo lo invitamos a visualizar el siguiente video:

4.3.4 Coeficiente de Jaccard: Conjuntos, Intersección y Unión

Cada abstract se reduce al **conjunto** de sus términos distintos (con stemming), sin importar cuántas veces aparezca cada uno:

- |A| = 76 términos y |B| = 52 términos.
- **A ∩ B** (se recorre el conjunto más pequeño, B, y se consulta cada término en A) = { artifici, intellig, educ, ai, relev, literatur, technologi, learn, understand, us, content, result, current, identifi, futur }, es decir, |A ∩ B| = 15.
- **|A ∪ B|** por inclusión-exclusión = |A| + |B| − |A ∩ B| = 76 + 52 − 15 = 113.

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|} = \frac{15}{113} = 0.1327$$

Las 15 palabras comunes son las mismas que aportan al producto punto TF-IDF (sección 4.3.3). Jaccard las cuenta todas igual, mientras que TF-IDF les da más peso a las raras ("content") y casi ninguno a las universales ("artifici").

Para una mejor comprensión lo invitamos a visualizar el siguiente video donde se mostrará como se halla la similitud basada en la teoría de conjuntos, le recordamos cambiar la calidad del video a 1080p: Video 6 Algoritmo de Jaccard

4.3.5 Los Dos Algoritmos de IA: Embeddings GloVe y FastText

Los dos algoritmos siguen los mismos pasos. Lo que cambia es el modelo que genera los vectores de palabra.

1. Tokens (sin stemming, sección 3.2): A = since, artificial, intelligence, ai, finding, way, areas, everyday, … (97 tokens) y B = applications, artificial, intelligence, education, aied, emerging, new, researchers, … (70 tokens).

2. Búsqueda de cada palabra en el modelo.

Tabla 10

Palabras de Cada Abstract con Vector en Cada Modelo

	GloVe 100d	FastText 300d
Palabras de A con vector	97 / 97	97 / 97
Palabras de B con vector	64 / 70 (faltan *aied* y *practitioners*)	**70 / 70**

Nota. Elaboración propia.

FastText encuentra *practitioners* y la sigla *AIED* (a partir de su variante en mayúsculas). Con su vocabulario de 1 millón de palabras cubre todo el abstract B; GloVe tiene que omitir 6 ocurrencias.

Ejemplo de las 5 primeras dimensiones de algunos vectores de palabra en cada modelo:

Tabla 11

Primeras Cinco Dimensiones de Algunos Vectores de Palabra

Palabra	GloVe d1	d2	d3	d4	d5	… d100	FastText d1	d2	d3	d4	d5	… d300
artificial	0.3455	0.3144	−0.0313	0.6368	0.2727	…	0.1049	0.0499	−0.0307	0.0107	0.0031	…
intelligence	−0.3110	−0.4329	0.7773	−0.3112	0.0529	…	0.0605	−0.0863	−0.0197	0.0831	−0.0720	…

Palabra	GloVe d1	d2	d3	d4	d5	… d100	FastText d1	d2	d3	d4	d5	… d300
literacy	0.1881	1.3096	−0.5283	0.6446	−0.1079	…	−0.0940	0.0278	0.1109	0.0162	−0.1208	…
education	0.2958	0.7682	−0.6430	0.3542	0.4718	…	0.0839	−0.0602	0.1342	−0.1375	−0.0973	…
students	0.7067	0.8647	−0.8373	0.1544	−0.4944	…	−0.0855	0.0609	0.0099	−0.0561	−0.0613	…

Nota. Elaboración propia.

Las dimensiones de un modelo no tienen relación con las del otro: cada entrenamiento produce su propio espacio vectorial. Por eso los vectores no se mezclan y cada algoritmo compara A y B dentro de su propio espacio. Lo que sí se puede comparar es lo que cada modelo aprendió. El coseno entre pares de palabras muestra, por ejemplo:

Tabla 12

Similitud del Coseno entre Pares de Palabras en Cada Modelo

Par de palabras	GloVe	FastText
teacher / student	0.8083	0.7072
assessment / evaluation	0.8064	0.7231
learning / education	0.6989	0.6378
literacy / skills	0.5586	0.5413
artificial / intelligence	0.2134	**0.5397**

Nota. Elaboración propia.

Los dos modelos coinciden en las relaciones del dominio educativo (*teacher*/*student*, *assessment*/*evaluation*), con valores algo menores en FastText. Pero también aprendieron relaciones distintas: para FastText *artificial* e *intelligence* son mucho más cercanas (0.54) que para GloVe (0.21). Son dos espacios vectoriales diferentes, y por eso cuentan como dos enfoques.

3. Vector de cada documento = promedio componente a componente de los vectores encontrados:

Tabla 13

Primeras Cinco Dimensiones del Vector Promedio de Cada Abstract

	GloVe d1	d2	d3	d4	d5	FastText d1	d2	d3	d4	d5
V_A (promedio de 97)	−0.0435	0.2915	0.1569	0.2096	−0.0006	−0.0266	−0.0032	−0.0287	−0.0212	−0.0139
V_B (64 en GloVe, 70 en FastText)	−0.0720	0.3226	0.0791	0.3260	0.1884	−0.0292	0.0188	−0.0275	−0.0241	−0.0082

Nota. Elaboración propia.

4. Similitud del coseno. Por ejemplo, los 5 primeros términos del producto punto en GloVe son (−0.0435)(−0.0720) + (0.2915)(0.3226) + (0.1569)(0.0791) + (0.2096)(0.3260) + (−0.0006)(0.1884) = 0.1778. En FastText suman 0.0021. Con todas las dimensiones:

GloVe (d = 100): $V_A \cdot V_B = 9.924754$ $\|V_A\| = 3.132159$ $\|V_B\| = 3.295900$

$$\cos\theta = \frac{9.924754}{(3.132159 \cdot 3.295900)} = 0.9614 \quad \theta = 15.97°$$

FastText (d = 300): $V_A \cdot V_B = 1.370037$ $\|V_A\| = 1.172698$ $\|V_B\| = 1.214720$

$$\cos\theta = \frac{1.370037}{(1.172698 \cdot 1.214720)} = 0.9618 \quad \theta = 15.89°$$

5. Distancia euclidiana (en el detalle de cada resultado):

GloVe: $\sum (V_A_i - V_B_i)^2 = 0.823870$ dist = sqrt(0.823870) = 0.9077 $\frac{1}{(1 + dist)} = 0.5242$

FastText: $\sum (V_A_i - V_B_i)^2 = 0.110691$ dist = sqrt(0.110691) = 0.3327 $\frac{1}{(1 + dist)} = 0.7504$

Las distancias no son comparables entre modelos, porque los vectores de FastText tienen normas mucho menores (≈ 1.2 frente a ≈ 3.2). El coseno, en cambio, sí lo es: los dos modelos ven a A y B separados por un ángulo de ≈ 16°.

Para mayor claridad sobre como se construyen los vectore y cómo calcular la similitud del coseno entre ellos lo invitamos a ver el siguiente video: Video 7 Algoritmos de IA

4.3.6 Resumen de Resultados del Artículo 1 frente al Artículo 2

Tabla 14

Resumen de la Similitud entre los Artículos 1 y 2

Algoritmo	Similitud	Valor clave del cálculo
Levenshtein (clásico)	**0.3206**	D[1129][887] = 767
Needleman-Wunsch (clásico)	**0.1237**	F[97][70] = −61 (13 coincidencias, 54 sustituciones, 33 gaps)
Coseno TF-IDF (clásico)	**0.1992**	producto punto 0.018316, θ = 78.51°
Jaccard (clásico)	**0.1327**	15 / 113
Embeddings GloVe + coseno (IA)	**0.9614**	θ = 15.97°, distancia euclidiana 0.9077

Algoritmo	Similitud	Valor clave del cálculo
Embeddings FastText + coseno (IA)	**0.9618**	θ = 15.89°, distancia euclidiana 0.3327

Nota. Elaboración propia.

La Tabla 14 evidencia una marcada brecha entre los algoritmos tradicionales y los enfoques basados en inteligencia artificial al evaluar la similitud entre el Artículo 1 y el Artículo 21. Los métodos clásicos reportan valores bajos (entre 0.1237 y 0.3206) debido a que se limitan a medir coincidencias léxicas exactas, alineamiento estricto de secuencias o la intersección literal de términos12. En contraste, los modelos de embeddings (GloVe con 0.9614 y FastText con 0.9618) capturan la alta proximidad semántica y contextual de ambos abstracts dentro del dominio educativo, separando sus vectores documento por un ángulo espacial muy cerrado ($\theta \approx 16^{\circ}$)

Para interpretar el caso de estudio hay que ver cómo se reparten los valores de cada algoritmo en los **190 pares** del corpus y en qué posición queda el par 1–2:

Tabla 15

Distribución de la Similitud de los 190 Pares del Corpus por Algoritmo

Algoritmo	Mínimo	Máximo	Promedio	Desv. est.	Coef. de variación	Correlación con TF-IDF	Puesto del par 1–2 (de 190)
Levenshtein	0.2334	0.3206	0.2819	0.0206	0.073	0.340	**1**
Needleman-Wunsch	0.0160	0.1237	0.0648	0.0220	0.340	0.740	**1**
Coseno TF-IDF	0.0265	0.2955	0.1384	0.0592	**0.428**	1	33
Jaccard	0.0385	0.2039	0.1116	0.0303	0.271	0.515	42
Embeddings GloVe + coseno	**0.9136**	**0.9833**	0.9551	0.0147	**0.015**	0.532	67
Embeddings FastText + coseno	**0.9397**	**0.9845**	0.9614	0.0089	**0.009**	0.531	91

Algoritmo	Mínimo	Máximo	Promedio	Desv. est.	Coef. de variación	Correlación con TF-IDF	Puesto del par 1–2 (de 190)
(detalle) **GloVe, 1/(1+dist. euclidiana)**	0.4348	0.6260	0.5191	0.0422	0.081	0.449	79
(detalle) **FastText, 1/(1+dist. euclidiana)**	0.7054	0.8259	0.7513	0.0225	0.030	0.504	89

Nota. Elaboración propia.

Estos datos de la tabla no se ven en el proyecto

(Coeficiente de variación = desviación estándar / promedio: mide qué tanto separa el algoritmo unos pares de otros, independientemente de su escala.)

1. Los dos enfoques de IA casi no discriminan.

Con GloVe los 190 pares quedan entre 0.914 y 0.983, y con FastText entre 0.940 y 0.985 (coeficientes de variación de 0.015 y 0.009, los más bajos de todos). El ≈ 0.96 del caso de estudio está muy cerca del promedio de cada modelo: el par 1–2 queda en el puesto 67 con GloVe y en el 91 con FastText. Por lo tanto, un coseno de 0.96 **no** significa "estos dos artículos tratan el mismo tema", porque casi cualquier par del corpus obtiene un valor así.

La causa es el promedio de vectores de palabra. Al promediar ~80 palabras, lo específico de cada texto se diluye y todos los vectores de documento apuntan hacia una dirección común, la del "inglés académico". En las primeras dimensiones de V_A y V_B se ve: tienen el mismo

signo y una magnitud parecida en ambos modelos. Además, los 20 artículos son revisiones sobre IA en educación, un corpus temáticamente homogéneo.

2. Más vocabulario no resolvió el problema.

FastText cubre todo el abstract B (70/70, frente a 64/70 de GloVe) y tiene 300 dimensiones en lugar de 100, pero el resultado del par 1–2 casi no cambia (0.9618 frente a 0.9614) y su dispersión es incluso menor. Los dos modelos coinciden bastante en **cómo ordenan** los pares (correlación de Pearson de 0.813 entre ambos sobre los 190 pares), así que el efecto viene del promedio de vectores, no del modelo. Los embeddings sí capturan cercanía semántica (*teacher*/*student* ≈ 0.7–0.8 sin compartir letras), pero para comparar documentos su resultado se debe leer **en términos relativos** (ranking) y no como valor absoluto.

3. La distancia euclidiana separa un poco más que el coseno en ambos modelos (coeficiente de variación 0.081 frente a 0.015 en GloVe; 0.030 frente a 0.009 en FastText) porque, además de la dirección, mide la diferencia de magnitud entre los vectores promedio. Esa magnitud depende de cuántas palabras específicas tiene cada abstract.

4. TF-IDF es el que mejor distingue los pares de este corpus (coeficiente de variación 0.428, el mayor). El idf anula el aporte de las palabras que están en todos los abstracts ("artifici", "intellig": idf = 1) y premia las que caracterizan a cada uno, que es justo lo que importa en un corpus donde todos hablan de lo mismo. Coloca al par 1–2 en el puesto 33, una similitud

media-alta: ambas son revisiones de literatura sobre IA en educación, pero una trata la alfabetización en IA de adultos y la otra un mapeo de oportunidades y desafíos.

5. Las distancias de edición miden forma, no tema. Levenshtein y Needleman-Wunsch ponen al par 1–2 en el **puesto 1** de 190, por encima de pares temáticamente más cercanos según TF-IDF (por ejemplo 8–10). Los dos abstracts mencionan "artificial intelligence" en sus primeras palabras y tienen la redacción típica de una revisión de literatura, y estos algoritmos premian el **orden** en que aparecen las palabras o letras. Levenshtein es el caso extremo: su rango es estrecho (0.23–0.32) porque dos textos cualesquiera en inglés comparten espacios, artículos y terminaciones ("-tion", "-ing"), y su correlación con TF-IDF es la más baja (0.34). A nivel de token, Needleman-Wunsch elimina parte de ese ruido y su correlación con TF-IDF sube a 0.74.

6. Jaccard y TF-IDF coinciden en las palabras pero no en los pesos. Ambos usan los mismos 15 términos comunes, pero Jaccard los cuenta todos igual. Por eso su correlación con TF-IDF (0.515) es moderada y ubica el par en el puesto 42.

Conclusión del caso de estudio.

- Para **encontrar artículos temáticamente relacionados** dentro de este corpus, el algoritmo más útil resultó ser el clásico **coseno TF-IDF**, porque separa mejor los pares.
- Los **dos enfoques de embeddings** (GloVe y FastText) capturan semántica y coinciden entre sí. Pero, al promediar los vectores de palabra, en un corpus homogéneo dan valores

casi iguales para todos los pares. Sirven para ordenar (qué par es más parecido que otro), no para decidir con un umbral absoluto si dos abstracts "son similares". Un modelo que represente la oración completa (*sentence embeddings*) o restar el vector promedio del corpus reduciría ese efecto.

- Las **distancias de edición** (Levenshtein, Needleman-Wunsch) responden a otra pregunta: si dos textos están escritos de forma parecida (detección de plagio, abstracts casi duplicados o versiones de un mismo texto), no si tratan el mismo tema.

Los clásicos miden coincidencia literal, y los modelos de IA miden cercanía semántica pero necesitan interpretarse de forma relativa.

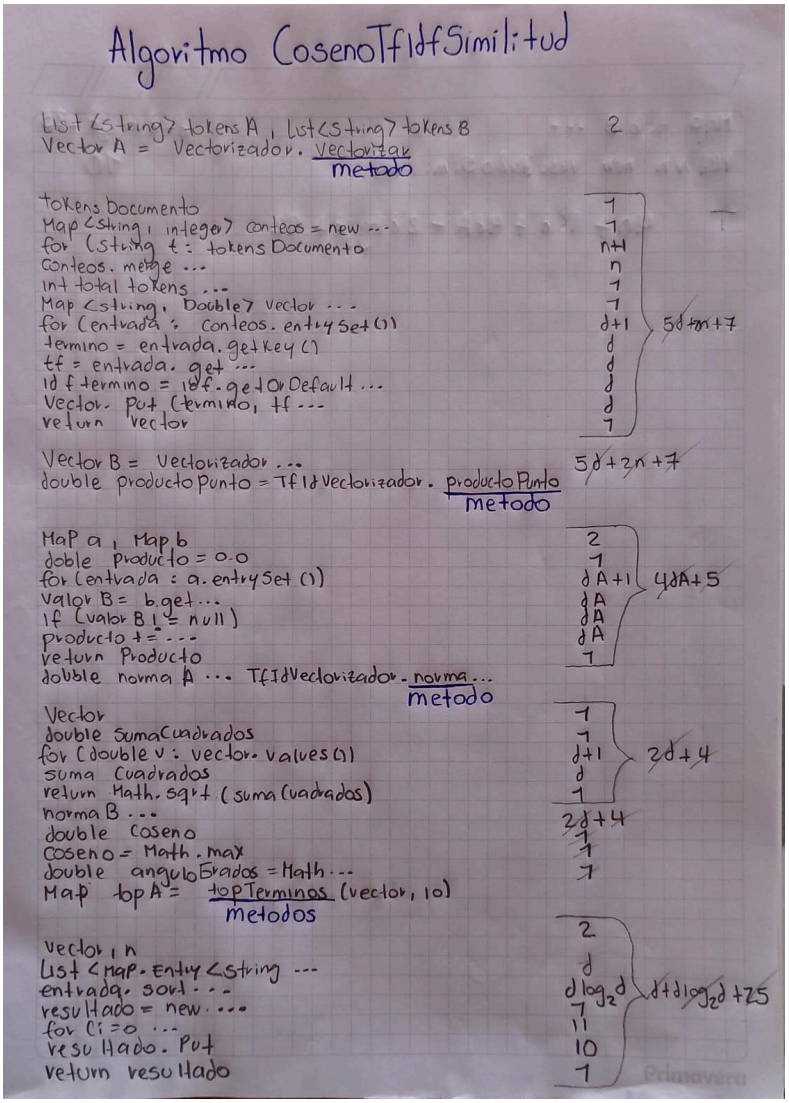
4.4 Análisis Formal de Complejidad Computacional

4.4.1 Similitud del coseno con TF-IDF

Para hallar el tiempo de ejecución T(n) del algoritmo se analizó el método calcular de la clase cosenoTfIdfSimilitud línea por línea, junto con los métodos que invoca: vectorizar.productoPunto, norma y topTerminos. A cada línea se le asignó el número de veces que se ejecuta. Los ciclos for cuentan una ejecución adicional, correspondiente a la comparación de salida. Las operaciones sobre tablas hash (get, put, merge) se consideran de costo constante. Se definen las variables n como el número de tokens del abstract más largo y d como el número de términos distintos del abstract más largo, de modo que $d \leq n$. El análisis se presenta en la Figura 3

Figura 3

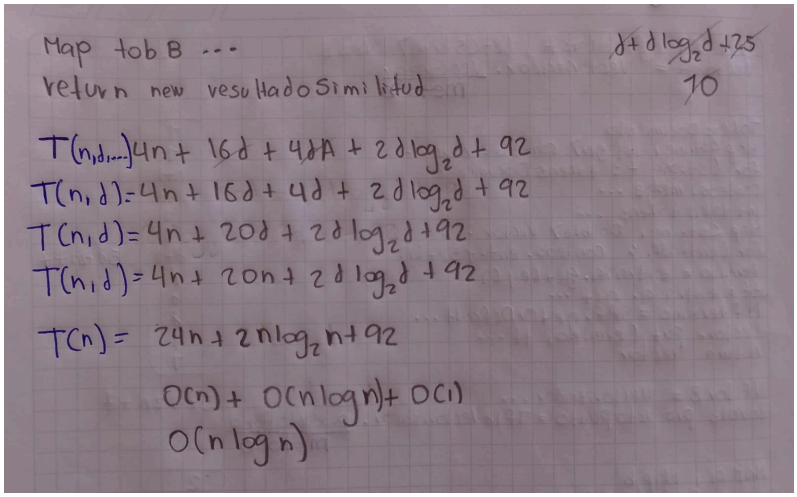
Tiempo de Ejecución del Algoritmo con su Complejidad Computacional



Nota. Elaboración propia.

Figura 4

Tiempo de Ejecución del Algoritmo con su Complejidad Computacional



Nota. Elaboración propia.

Al sumar los costos de todas las líneas se obtiene:

$$T = 4n + 16d + 4dA + 2d \cdot \log_2 d + 92$$

Como los términos distintos del abstract A no superan a los del abstract más largo (dA ≤ d), en el peor caso dA = d:

$$T(n, d) = 4n + 20d + 2d \cdot \log_2 d + 92$$

Al eliminar los términos de orden inferior (20d y 92) y las constantes multiplicativas, se obtiene:

$$T(n, d) \in O(n + d \log d)$$

El término n corresponde al recorrido de los tokens en vectorizar. El término d log d corresponde al ordenamiento de los términos en topTerminos. En el peor caso ninguna palabra se repite, así que d = n:

$$T(n) = 2n \cdot \log_2 n + 24n + 92 \in O(n \log n)$$

El cálculo de la similitud en sí (vectorización, producto punto, normas y coseno) es lineal. El término n log n proviene del ordenamiento que se hace para mostrar los diez términos de mayor peso en la interfaz. Para el caso de estudio (n = 97), T(97) ≈ 3700 operaciones.

4.4.2 Coeficiente de Jaccard

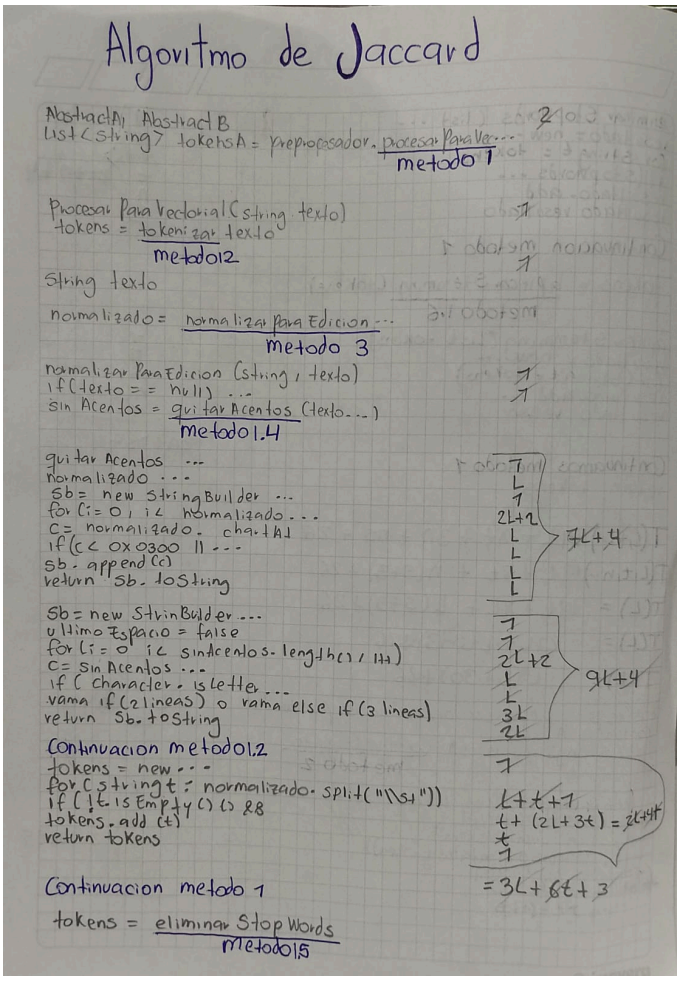
Para hallar el tiempo de ejecución T(n) del coeficiente de Jaccard se analizó línea por línea el método calcular de la clase JaccardSimilitud, junto con los métodos que invoca. A diferencia del coseno TF-IDF, este método recibe el texto crudo de los abstracts, por lo que el análisis incluye el preprocesamiento completo. El método procesarParaVectorial de la clase Preprocesador invoca a su vez a tokenizar, normalizarParaEdicion, quitarAcentos, eliminarStopwords y aplicarStemming. Después de preprocesar, calcular invoca a construirConjunto para cada abstract.

Se definen las variables:

- L: número de caracteres del abstract más largo.
- t: número de tokens después de tokenizar.
- n: número de tokens después de eliminar las stopwords.
- d: número de palabras distintas.
- k: número de palabras distintas del conjunto más pequeño.
- S: costo del stemming por palabra. Es constante, porque el algoritmo de Porter aplica cinco pasos fijos con listas de sufijos fijas.

Figura 5

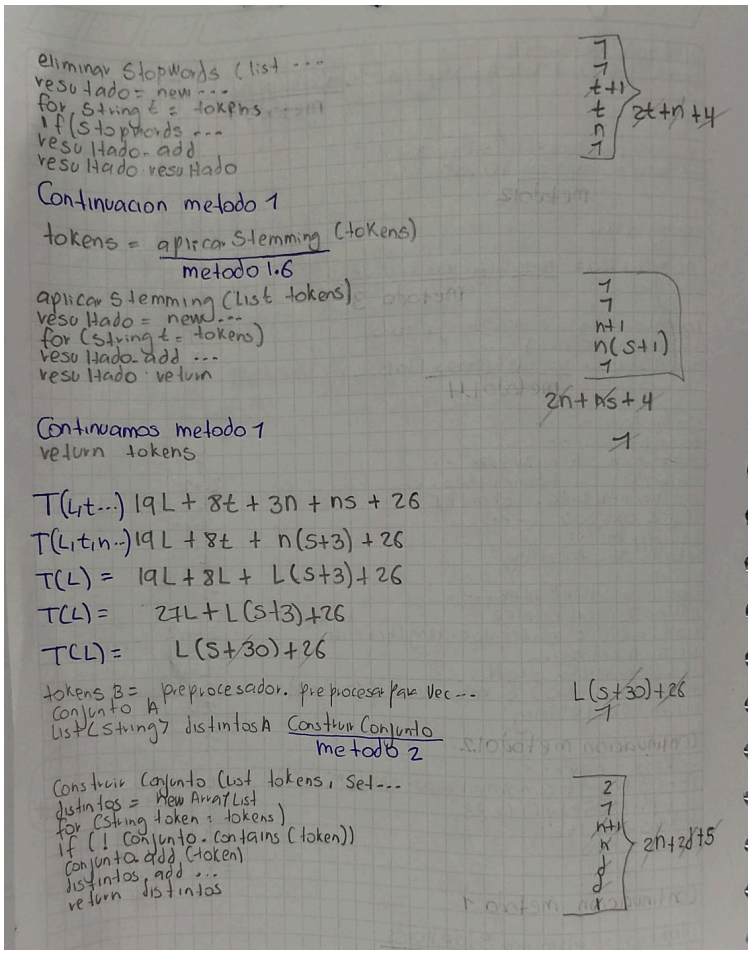
Tiempo de Ejecución del Algoritmo con su Complejidad Computacional



Nota. Elaboración propia.

Figura 6

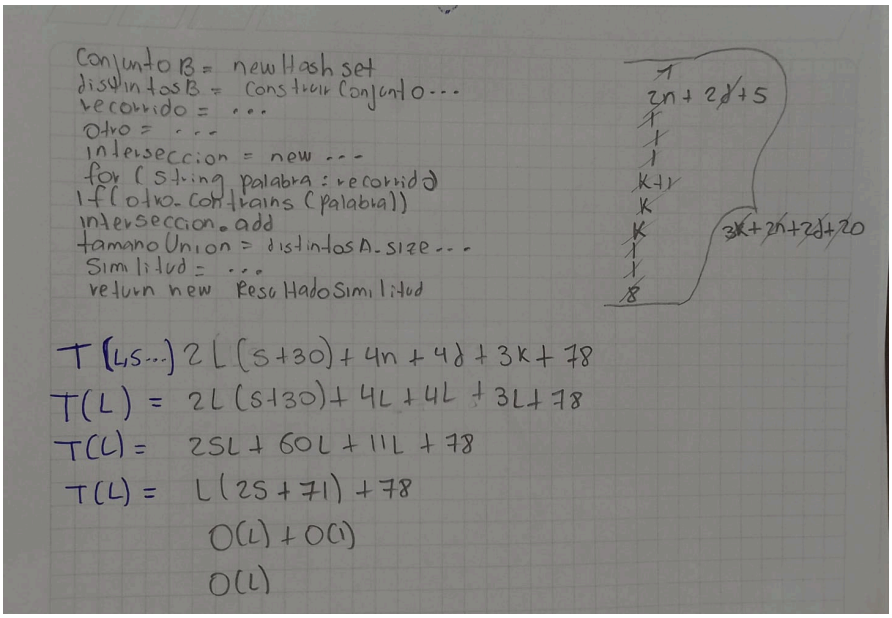
Tiempo de Ejecución del Algoritmo con su Complejidad Computacional



Nota. Elaboración propia.

Figura 7

Tiempo de Ejecución del Algoritmo con su Complejidad Computacional



Nota. Elaboración propia.

El coeficiente de Jaccard tiene complejidad lineal. El término dominante es el preprocesamiento de los dos abstracts, que recorre el texto una cantidad fija de veces. La construcción de los conjuntos y la intersección recorren los tokens una sola vez, con consultas de costo constante en la tabla hash. La unión no requiere otro recorrido, porque se obtiene por inclusión-exclusión en ($|A \cup B| = |A| + |B| - |A \cap B|$).

4.4.3 Distancia de Levenshtein

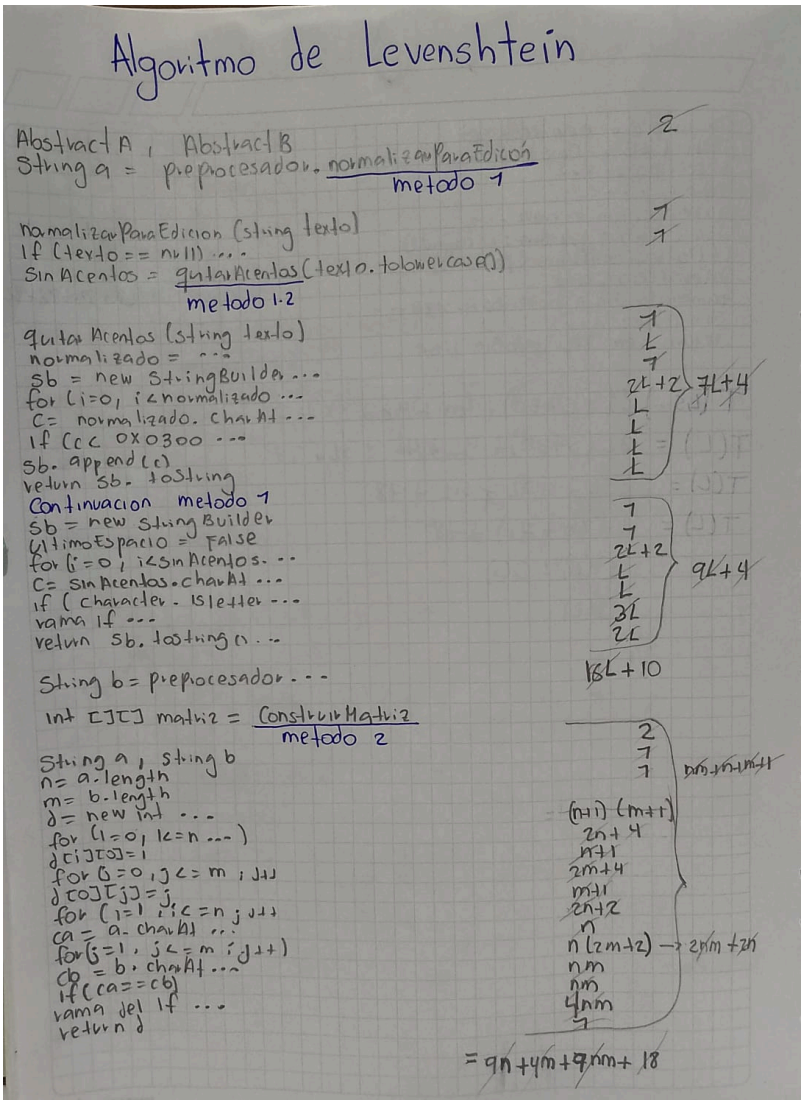
Para hallar el tiempo de ejecución T(n) de la distancia de Levenshtein se analizó línea por línea el método calcular de la clase LevenshteinSimilitud, junto con los métodos que invoca. Primero, cada abstract se normaliza con el método normalizarParaEdicion de la clase Preprocesador, que a su vez invoca a quitarAcentos. A diferencia de Jaccard, no se tokeniza ni se eliminan stopwords, porque Levenshtein compara el texto carácter por carácter. Después se construye la matriz de programación dinámica con el método construirMatriz.

Se definen las variables:

- L: número de caracteres del abstract más largo, antes de normalizar.
- n: número de caracteres de la cadena A normalizada.
- m: número de caracteres de la cadena B normalizada.

Figura 8

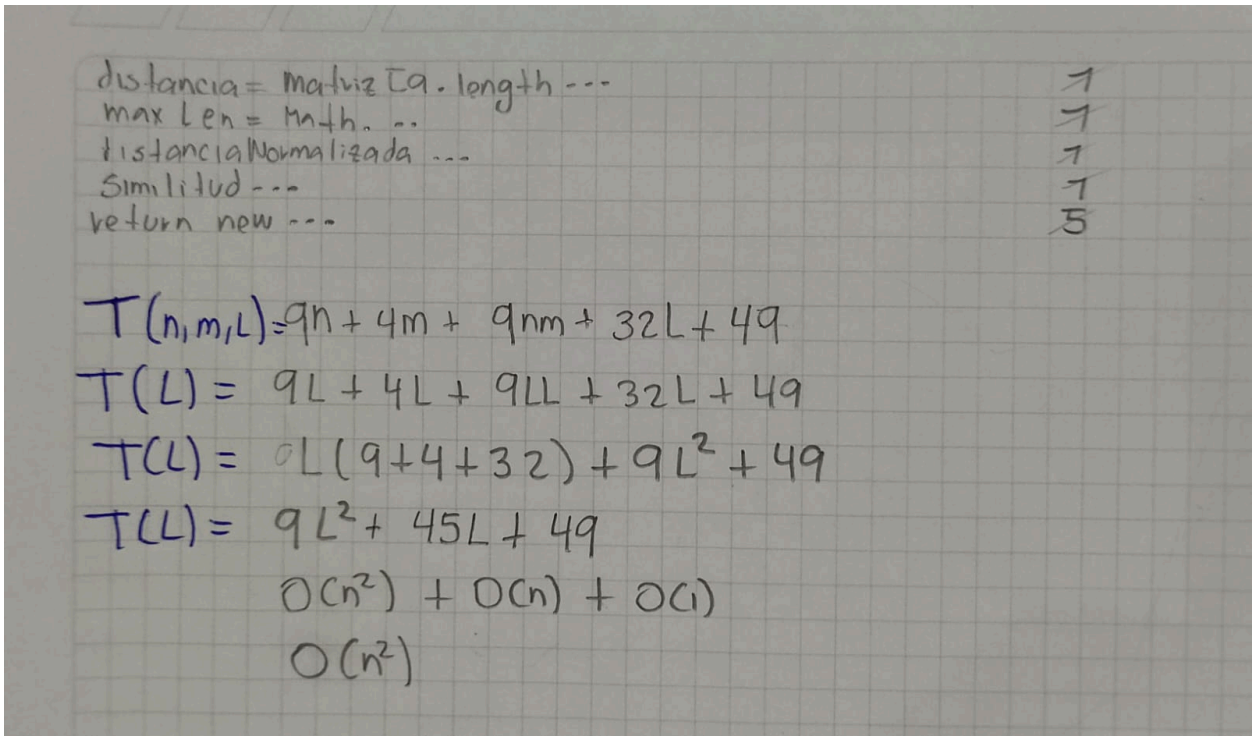
Tiempo de Ejecución del Algoritmo con su Complejidad Computacional



Nota. Elaboración propia.

Figura 9

Tiempo de Ejecución del Algoritmo con su Complejidad Computacional



Nota. Elaboración propia.

4.4.4 Alineamiento de Needleman-Wunsch

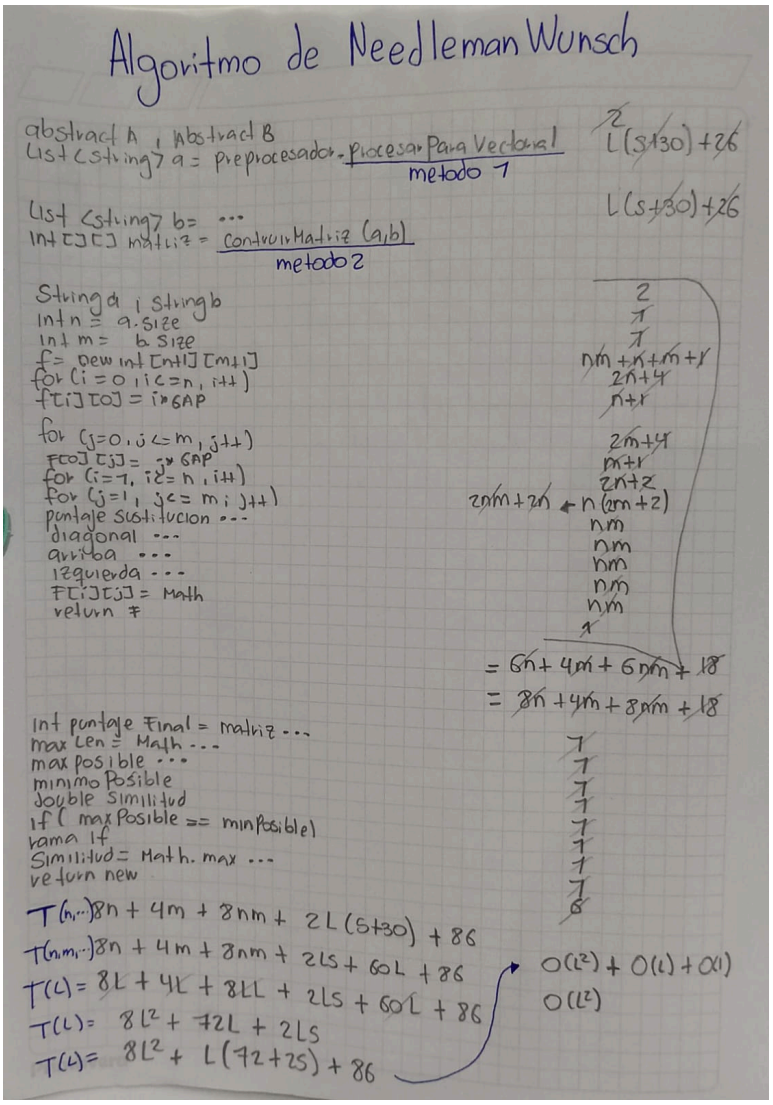
Para hallar el tiempo de ejecución T(n) del algoritmo de Needleman-Wunsch se analizó línea por línea el método calcular de la clase NeedlemanWunschSimilitud, junto con los métodos que invoca. Primero, cada abstract se preprocesa con el método procesarParaVectorial de la clase Preprocesador, cuyo tiempo de ejecución ya se obtuvo en el análisis del coeficiente de Jaccard. Después se construye la matriz de programación dinámica con el método construirMatriz. A diferencia de Levenshtein, el alineamiento se realiza sobre secuencias de palabras (tokens) y no de caracteres.

Se definen las variables:

- L: número de caracteres del abstract más largo.
- n: número de tokens del abstract A después del preprocesamiento.
- m: número de tokens del abstract B después del preprocesamiento.
- S: costo del stemming por palabra, que es constante.

Figura 10

Tiempo de Ejecución del Algoritmo con su Complejidad Computacional



Nota. Elaboración propia.

Al sumar los costos de todas las líneas se obtiene:

$$T(L, n, m) = 8nm + 2L(S + 30) + 8n + 4m + 86$$

El término 8nm corresponde al llenado de la matriz; 2L(S + 30), al preprocesamiento de los dos abstracts; 8n y 4m, a los ciclos que inicializan y recorren las filas y columnas; y 86, a las líneas que se ejecutan una sola vez.

En el peor caso ninguna palabra es eliminada durante el preprocesamiento, por lo que n = m = L:

$$T(L) = 8L^2 + 2LS + 60L + 8L + 4L + 86$$
$$T(L) = 8L^2 + L(72 + 2S) + 86$$

Al eliminar los términos de orden inferior, L(72 + 2S) y 86, y la constante multiplicativa 8, se obtiene:

$$T(L) \in O(L^2)$$

En términos de las longitudes de las dos secuencias de tokens, la complejidad es **O(n · m)**, que corresponde al tamaño de la matriz de programación dinámica.

5. Requerimiento 2: Agrupamiento Jerárquico

Se construye un árbol dendrograma que representa la similitud entre los 20 abstracts mediante cuatro etapas. Cada una tiene su sección:

Tabla 16

Etapas del Requerimiento 2 y Sección donde se Desarrollan

Etapa	Qué se hace	Implementación	Sección
1. Preprocesamiento	Transformar cada abstract en un vector TF-IDF	`texto.Preprocesador`, `TfIdfVectorizador`	5.1
2. Cálculo de la similitud	Matriz 20 × 20 de distancias d = 1 − coseno(TF-IDF)	`clustering.DistanciaCorpus`	5.2
3. Aplicación del clustering	Algoritmo aglomerativo con 3 criterios de enlace	`ClusteringJerarquico` + `EnlaceSimple` / `EnlaceCompleto` / `EnlacePromedio`	5.3
4. Representación	Un dendrograma por criterio	`NodoDendrograma`, interfaz web, sección 5.4	5.4
Evaluación	Métricas internas: correlación cofenética y silueta	`evaluacion.CorrelacionCofenetica`, `evaluacion.Silueta`, `CorteDendrograma`	5.5

Nota. Elaboración propia.

Todos los valores de esta sección se generaron con el código real (`app/src/test/java/.../GenerarCasoClustering.java`) y son los mismos que muestra la pestaña "Requerimiento 2" de la aplicación.

5.1 Preprocesamiento del Texto

Cada abstract pasa por el pipeline `procesarParaVectorial`. Ejemplo con la primera oración del artículo 1:

Tabla 17

Transformación del Abstract del Artículo 1 en Cada Paso del Preprocesamiento

Paso	Resultado
Original	*Since artificial intelligence (AI) is finding its way into more and more areas of everyday life, improving the AI skills of non-experts is important and will become even more relevant in the future.*
1. Minúsculas, sin acentos ni puntuación	since artificial intelligence ai is finding its way into more and more areas of everyday life improving the ai skills of non experts is important and will become even more relevant in the future
2. Tokenización	34 tokens: since, artificial, intelligence, ai, is, finding, its, way, into, more, and, more, areas, of, …
3. Sin stopwords	20 tokens: since, artificial, intelligence, ai, finding, way, areas, everyday, life, improving, ai, skills, non, experts, important, will, become, even, relevant, future
4. Stemming (Porter)	sinc, artifici, intellig, ai, find, wai, area, everydai, life, improv, ai, skill, non, expert, import, will, becom, even, relev, futur

Nota. Elaboración propia.

Con el abstract completo, el artículo 1 queda en 97 tokens y 76 términos distintos. Cada término recibe su peso TF-IDF, calculado con el idf de los 20 abstracts, como se mostró paso a paso en la sección 4.3.3. El resultado de esta etapa es un vector disperso por artículo.

5.2 Cálculo de la Similitud: Matriz de Distancias

Para cada uno de los C(20, 2) = 190 pares se calcula `distancia = 1 − coseno(TF-IDF)` (sección 4.1.3) y se arma una matriz simétrica 20 × 20 con diagonal 0. Se eligió TF-IDF porque en el Requerimiento 1 resultó la medida que mejor distingue los pares del corpus. En cambio, los embeddings promediados dan valores casi iguales para todos los pares, y con ellos el clustering no tendría estructura que mostrar.

Fragmento real de la matriz con los 8 artículos que intervienen en las primeras fusiones:

Tabla 18

Fragmento de la Matriz de Distancias

	[8]	[10]	[11]	[17]	[2]	[6]	[9]	[12]
[8]	0	**0.7045**	0.8504	0.8648	0.7686	0.8901	0.8361	0.8622
[10]	**0.7045**	0	0.8143	0.7933	0.7485	0.8487	0.8315	0.8949
[11]	0.8504	0.8143	0	**0.7099**	0.7539	0.8740	0.8069	0.8248
[17]	0.8648	0.7933	**0.7099**	0	0.7596	0.8338	0.8078	0.8682
[2]	0.7686	0.7485	0.7539	0.7596	0	**0.7233**	0.7663	0.8199
[6]	0.8901	0.8487	0.8740	0.8338	**0.7233**	0	0.7291	0.8203
[9]	0.8361	0.8315	0.8069	0.8078	0.7663	0.7291	0	0.7885
[12]	0.8622	0.8949	0.8248	0.8682	0.8199	0.8203	0.7885	0

Nota. Elaboración propia.

[8] *AI techniques for collaborative learning* y [10] *AI in simulation-based learning* son el par más cercano del corpus (0.7045). Les siguen [11] y [17], las dos revisiones sobre IA y evaluación (0.7099), y [2] y [6], dos panoramas generales de la IA en educación (0.7233).

Observación:

las 190 distancias están entre **0.7045 y 0.9735**, es decir, la similitud coseno más alta entre dos abstracts es apenas 0.2955. Los 20 artículos tratan el mismo dominio, pero cada uno con su propio vocabulario específico, y esto condiciona todos los resultados de las secciones siguientes. Para entender mejor este requerimiento dos lo invitamos a visualizar el siguiente video donde se abarcan los procesos previos para la construcción de la matriz con las distancias para posteriormente ser utilizada por los tres algoritmos recuerde poner la calidad del video en 1080p:

Video 8 construcción de la matriz de distancias

5.3 Aplicación del Clustering: Algoritmo Aglomerativo y Criterios de Enlace

Se implementó un único algoritmo aglomerativo (`clustering.ClusteringJerarquico`), desde cero y sin librerías:

1. Empieza con 20 clusters de un artículo cada uno, con las distancias de la matriz de la sección 5.2.
2. En cada iteración busca el par de clusters activos con **menor distancia** y los fusiona en uno nuevo.
3. Calcula la distancia del nuevo cluster I ∪ J a cada cluster restante M con la **fórmula de Lance-Williams** (Lance y Williams, 1967) del criterio elegido (patrón *Strategy*, interfaz `EstrategiaEnlace`).
4. Repite hasta que queda un solo cluster: 19 fusiones. Cada fusión es un nodo del dendrograma (`NodoDendrograma`), con la distancia a la que ocurrió como altura.

Tabla 19

Fórmulas de Lance-Williams para los Tres Criterios de Enlace

Enlace	Fórmula de actualización	Significado	Clase
Single Linkage	$d(I \cup J, M) = \min(d(I,M), d(J,M))$	distancia entre los dos artículos **más cercanos** de los clusters	`EnlaceSimple`
Complete Linkage	$d(I \cup J, M) = \max(d(I,M), d(J,M))$	distancia entre los dos artículos **más lejanos**	`EnlaceCompleto`
Average Linkage (UPGMA)	$d(I \cup J, M) = \frac{(\lvert I\rvert \cdot d(I,M) + \lvert J\rvert \cdot d(J,M))}{(\lvert I\rvert + \lvert J\rvert)}$	**promedio** de las distancias entre todos los pares de artículos	`EnlacePromedio`

Nota. Elaboración propia.

La fórmula de Lance-Williams evita recalcular la distancia entre clusters recorriendo todos sus pares de artículos: el valor se obtiene en O(1) a partir de las distancias que ya se tenían. Las pruebas¿ comparan las 19 alturas de fusión de cada criterio con una implementación ingenua que sí recorre todos los pares en cada paso, y coinciden exactamente.

5.3.1 Paso a Paso: las Primeras Fusiones

Tabla 20

Primeras Fusiones de Cada Criterio de Enlace

#	Single Linkage	Complete Linkage	Average Linkage
1	[8] + [10] a 0.7045	[8] + [10] a 0.7045	[8] + [10] a 0.7045
2	[11] + [17] a 0.7099	[11] + [17] a 0.7099	[11] + [17] a 0.7099
3	[2] + [6] a 0.7233	[2] + [6] a 0.7233	[2] + [6] a 0.7233
4	**[9] + [2, 6] a 0.7291**	**[12] + [20] a 0.7406**	**[12] + [20] a 0.7406**
5	[12] + [20] a 0.7406	[3] + [2, 6] a 0.7589	[9] + [2, 6] a 0.7477
6	[9, 2, 6] + [12, 20] a 0.7444	[16] + [11, 17] a 0.7766	[5] + [8, 10] a 0.7718

Nota. Elaboración propia.

Las tres primeras fusiones son iguales en los tres criterios. Mientras se comparan clusters de un solo artículo, el mínimo, el máximo y el promedio de una única distancia son el mismo número. Los criterios empiezan a diferir en la **fusión 4**, cuando hay que medir la distancia de [9] al cluster {2, 6} formado en la fusión 3:

Ejemplo de la actualización de Lance-Williams

con I = [2], J = [6] y M = [9], usando d([2],[9]) = 0.7663 y d([6],[9]) = 0.7291 de la matriz:

$$\begin{aligned}
&\text{Single:} && d(\{2,6\}, 9) = \min(0.7663,\ 0.7291) && = 0.7291\\
&\text{Complete:} && d(\{2,6\}, 9) = \max(0.7663,\ 0.7291) && = 0.7663\\
&\text{Average:} && d(\{2,6\}, 9) = \frac{(1 \cdot 0.7663 + 1 \cdot 0.7291)}{(1+1)} && = 0.7477
\end{aligned}$$

En la fusión 4 el algoritmo elige la menor distancia entre todos los clusters activos:

- **Single** obtiene 0.7291, menor que d([12],[20]) = 0.7406, y fusiona [9] con {2, 6}. Le basta que [9] esté cerca de **uno** de los dos ([6]).
- **Complete** obtiene 0.7663 y **Average** 0.7477, ambos mayores que 0.7406, así que primero fusionan [12] con [20]. Average une [9] con {2, 6} en la fusión 5, a exactamente 0.7477. Complete no lo hace hasta mucho después: [9] está lejos de [2], y Complete exige que **todos** los pares estén cerca.

El mismo cálculo para la primera fusión (I = [8], J = [10]) frente a otros clusters muestra cómo divergen los criterios desde el principio:

Tabla 21

Distancia del Cluster Fusionado a los Demás según Cada Criterio

M	d([8], M)	d([10], M)	Single (mín)	Complete (máx)	Average
[1]	0.8615	0.8561	0.8561	0.8615	0.8588
[2]	0.7686	0.7485	0.7485	0.7686	0.7586
[3]	0.8313	0.8653	0.8313	0.8653	0.8483
[4]	0.9112	0.8964	0.8964	0.9112	0.9038

Nota. Elaboración propia.

Single siempre da la distancia más baja y Complete la más alta. Por eso, para un mismo par de clusters, Single los fusiona antes y Complete después.

5.3.2 Las Últimas Fusiones: Cómo Termina Cada Árbol

Tabla 22

Últimas Fusiones de Cada Criterio de Enlace

#	Single Linkage	Complete Linkage	Average Linkage
17	[15] + grupo de 17 a 0.8313	[7] + {19, 18, 13, 16, 11, 17} a 0.9335	[19] + grupo de 17 a 0.9029
18	[4] + grupo de 18 a 0.8864	[4] + grupo de 12 a 0.9502	[7] + grupo de 18 a 0.9171
19	[7] + grupo de 19 a 0.8894	**grupo de 7 + grupo de 13** a 0.9735	[4] + grupo de 19 a 0.9331

Nota. Elaboración propia.

- **Single Linkage** termina **encadenando**: sus últimas fusiones agregan los artículos atípicos ([15], [4], [7]) uno por uno a un único cluster grande. Es el efecto conocido como *chaining*: basta un vecino cercano para entrar al grupo.
- **Complete Linkage** termina uniendo **dos grupos grandes** (7 y 13 artículos). Su última altura, 0.9735, es la distancia máxima de todo el corpus, la de [14] con [19], como debe ser: Complete mide la distancia entre los artículos más lejanos.
- **Average Linkage** también termina agregando artículos atípicos de uno en uno ([19], [7], [4]), aunque a alturas mayores que Single.

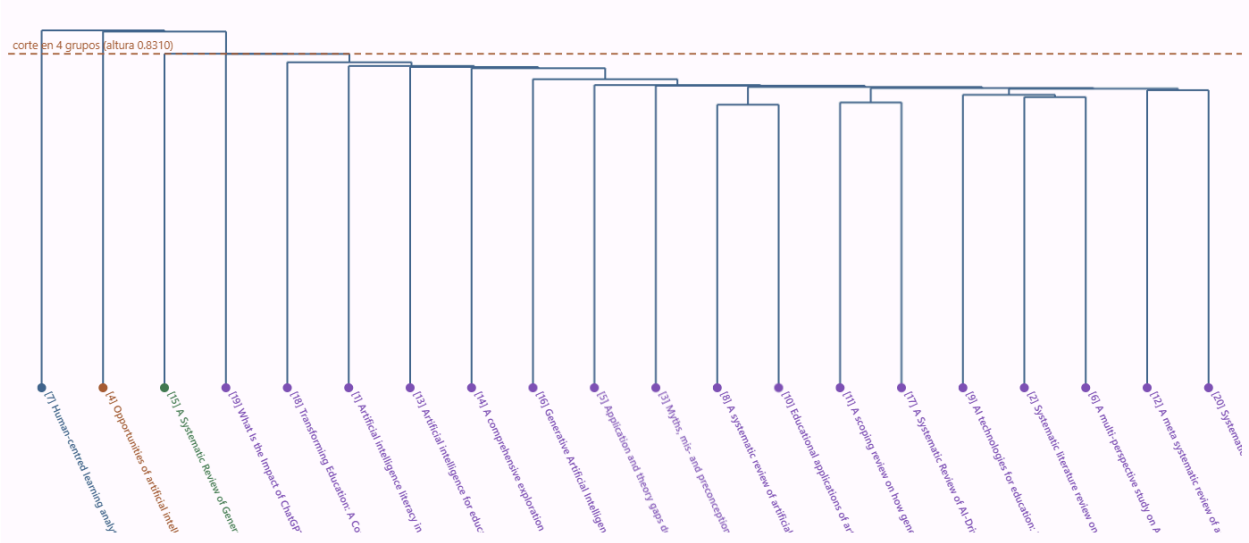
5.4 Representación Visual: los Tres Dendrogramas

Cada imagen muestra el dendrograma completo de un criterio. Cada U es una fusión, dibujada a la altura (distancia) en que ocurrió. Los colores indican los 4 grupos que resultan de

cortar el árbol en la línea punteada. El eje vertical empieza en 0.65 porque todas las fusiones ocurren entre 0.70 y 0.97. En la aplicación web los tres dendrogramas se dibujan en SVG (`app.js::construirSvgDendrograma`) a partir de `GET /api/clustering`, y se pueden cortar en 2 a 6 grupos con colores, la línea de corte y la lista de las 19 fusiones.

Figura 11

Dendrograma del Criterio Single Linkage

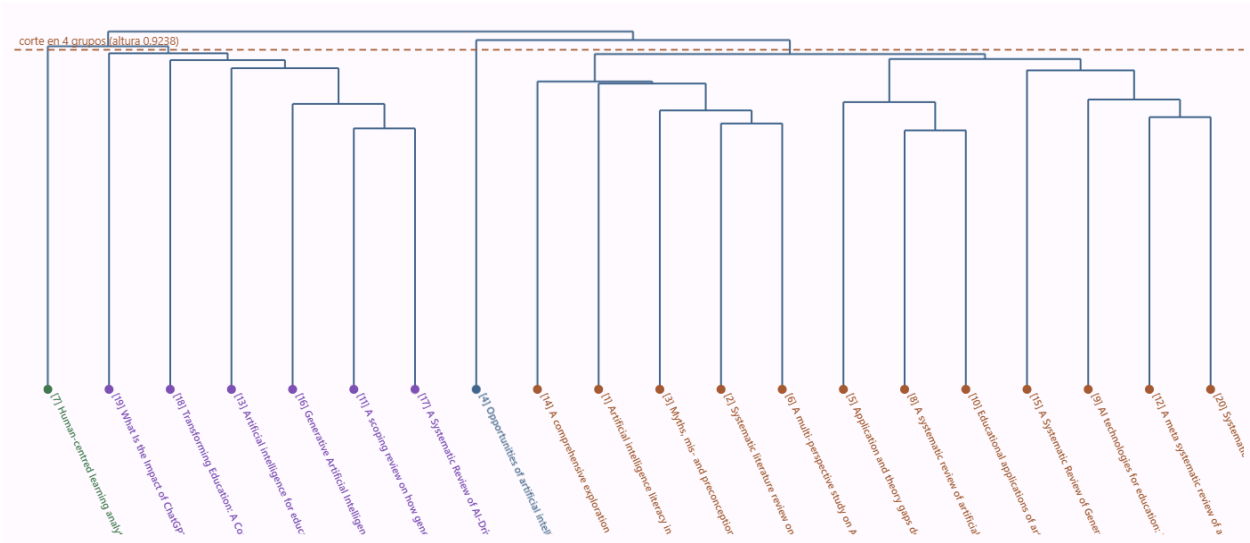


Nota. Elaboración propia.

Single Linkage (cofenética 0.7476). Forma una "escalera": un gran cluster al que se van uniendo los artículos uno por uno. Cortado en 4 grupos, deja tres artículos sueltos ([7], [4], [15]) y uno de 17.

Figura 12

Dendrograma del Criterio Complete Linkage

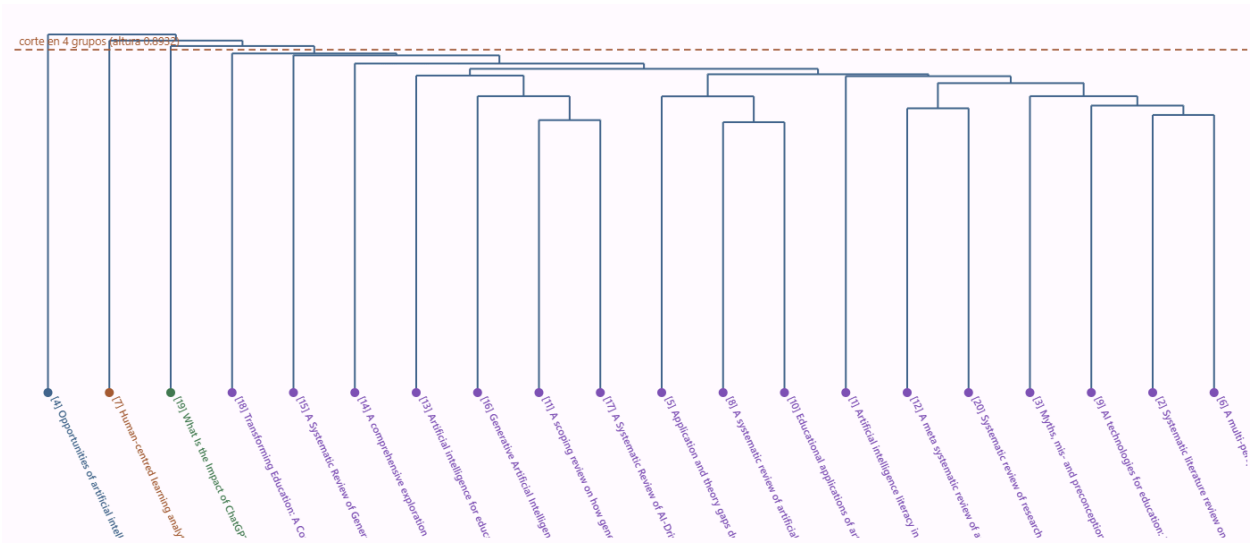


Nota. Elaboración propia.

Complete Linkage (cofenética 0.5717). Es el árbol más equilibrado. Cortado en 4 grupos da {11, 13, 16, 17, 18, 19}, un grupo grande de 12 artículos, y [4] y [7] sueltos. Sus ramas inferiores también forman subgrupos con sentido: {8, 10, 5}, {2, 6, 3}, {12, 20, 9, 15}.

Figura 13

Dendrograma del Criterio Average Linkage



Nota. Elaboración propia.

Average Linkage (cofenética 0.8021). Por dentro forma subgrupos parecidos a los de Complete, por ejemplo {11, 17, 16, 13} y {8, 10, 5}. Pero sus fusiones más altas separan de a uno los artículos atípicos ([4], [7], [19], [18], [15]), así que al cortarlo en pocos grupos se comporta como Single.

5.5 Evaluación Interna: ¿Qué Algoritmo Produce Agrupamientos más Coherentes?

Se usaron dos métricas internas, es decir, que no requieren una clasificación "correcta" conocida de los artículos. Miden aspectos distintos de la coherencia:

5.5.1 Métrica 1: Coeficiente de Correlación Cofenética

1. La **distancia cofenética** entre dos artículos es la altura del nodo donde el dendrograma los une por primera vez (`CorrelacionCofenetica.calcularMatrizCofenetica`, recorrido recursivo del árbol). Es la distancia que "dice" el árbol.
2. La **correlación cofenética** (Sokal y Rohlf, 1962) es el coeficiente de correlación de Pearson, implementado a mano, entre las 190 distancias originales y las 190 cofenéticas.

Un valor cercano a 1 indica que el árbol conserva fielmente las distancias originales.

Ejemplo con el árbol de Average Linkage:

Tabla 23

Distancias Originales y Cofenéticas de Algunos Pares con Average Linkage

Par	Distancia original	Distancia cofenética (altura donde se unen)	Comentario
[8]–[10]	0.7045	0.7045	Se unen en la primera fusión: el árbol conserva su distancia exacta.
[8]–[11]	0.8504	0.8435	Cercana a la original.
[1]–[2]	0.8008	0.8243	El árbol los separa un poco más de lo que están.
[14]–[19]	0.9735	0.9029	El par más lejano del corpus: el árbol lo une antes (más cerca) de lo real.
[4]–[11]	0.9540	0.9331	

Nota. Elaboración propia.

Tabla 24

Correlación Cofenética de los Tres Criterios de Enlace

Algoritmo de enlace	Correlación cofenética
Single Linkage	0.7476
Complete Linkage	0.5717
Average Linkage (UPGMA)	**0.8021**

Nota. Elaboración propia.

Según esta métrica, Average Linkage produce el dendrograma más coherente con las distancias originales. Es un resultado esperado en la teoría: UPGMA fusiona usando el promedio de todas las distancias entre los dos clusters, y es el criterio que tiende a maximizar la correlación cofenética (Farris, 1969). Complete queda último porque, al usar el par más lejano, sitúa sus fusiones altas muy por encima de la mayoría de las distancias que resume.

5.5.2 Métrica 2: Coeficiente de Silueta

La correlación cofenética evalúa el árbol completo, pero no dice si los grupos que se obtienen al cortarlo son compactos y están separados. Para eso se implementó el coeficiente de silueta (evaluacion.Silueta). Cada dendrograma se corta en k = 2 a 6 grupos (CorteDendrograma, deshaciendo las k − 1 fusiones más altas) y, para cada artículo i:

$$
\begin{aligned}
a(i) &= \text{distancia promedio de } i \text{ a los demás artículos de su grupo} \\
b(i) &= \text{menor distancia promedio de } i \text{ a los artículos de otro grupo} \\
s(i) &= \frac{(b(i) - a(i))}{\max(a(i), b(i))} \quad (0 \text{ si } i \text{ está solo en su grupo}) \\
S &= \text{promedio de } s(i) \text{ sobre los 20 artículos } \in [-1, 1]
\end{aligned}
$$

S cercano a 1 indica grupos compactos y bien separados; cercano a 0, que los artículos están en la frontera entre grupos. Kaufman y Rousseeuw (1990) consideran que S < 0.25 indica que no hay una estructura de grupos sustancial.

Tabla 25

Coeficiente de Silueta por Criterio y Número de Grupos

Algoritmo	k = 2	k = 3	k = 4	k = 5	k = 6
Single	0.066 [19, 1]	0.065 [18, 1, 1]	0.036 [17, 1, 1, 1]	0.034 [16, 1, 1, 1, 1]	0.025 [15, 1, 1, 1, 1, 1]
Complete	0.035 [13, 7]	0.037 [12, 7, 1]	0.038 [12, 6, 1, 1]	0.030 [12, 5, 1, 1, 1]	0.021 **[7, 5, 5, 1, 1, 1]**
Average	**0.081** [19, 1]	0.065 [18, 1, 1]	**0.050** [17, 1, 1, 1]	0.037 [16, 1, 1, 1, 1]	0.025 [15, 1, 1, 1, 1, 1]

Nota. Entre corchetes, el tamaño de cada grupo.

Para entender mejor esta sección del requerimiento dos lo invitamos a visualizar el siguiente video donde se abarcan el análisis matemático y ejecución de los 3 algoritmos de

aglomeramiento recuerde poner la calidad del video en 1080p: Video 9 Analisis matemático y ejecución de aglomeramiento

5.5.3 Interpretación y Conclusión

1. Ninguno de los tres criterios encuentra grupos bien separados. Todas las siluetas están entre 0.02 y 0.08, muy por debajo de 0.25. No es un defecto de los algoritmos: Los 20 artículos son revisiones de literatura sobre un mismo dominio, IA en educación, y no se dividen en temas claramente separados.
2. Single y Average producen la misma partición en cada corte. Ambos dejan un grupo grande y separan artículos atípicos de uno en uno. Esos artículos atípicos tienen el vocabulario más específico: [7] learning analytics, [4] complex problem-solving, [19] ChatGPT, [18] análisis bibliométrico, [15] IA generativa en la enseñanza. La silueta un poco mayor de Average en k = 2 (0.081) viene de separar un solo artículo, [4], no de formar grupos temáticos.

3. Complete es el único que forma grupos de tamaño comparable, y esos grupos tienen sentido temático, aunque numéricamente estén poco separados. Su corte en 6 grupos es:

6. Requerimiento 3: Búsqueda de Patrones

La aplicación permite escribir un patrón de búsqueda y localiza **todas sus ocurrencias exactas** en los 20 abstracts con tres algoritmos clásicos de *string matching*: Knuth-Morris-Pratt (KMP), Boyer-Moore y Rabin-Karp. Los tres se construyeron desde cero sobre arreglos `char[]`: solo usan accesos por índice y comparaciones de caracteres. No se usa `indexOf`, `contains`, `find`, `matches` ni expresiones regulares. Los tres devuelven exactamente las mismas posiciones: la aplicación lo valida en cada búsqueda y las pruebas de la sección 8 lo verifican contra un oráculo.

6.1 Diseño

El código está en el paquete `busqueda`. Cada algoritmo implementa la interfaz genérica `AlgoritmoBusqueda<E>`, que separa explícitamente las dos fases que el enunciado pide comparar:

Tabla 26

Métodos de la Interfaz AlgoritmoBusqueda

Método	Fase	Depende de
`E preprocesar(char[] patron)`	Construye la estructura auxiliar del patrón (tipo E)	solo m
`ResultadoBusqueda buscar(texto, patron, estructura, traza)`	Recorre el texto usando esa estructura	n (y m)

Nota. Elaboración propia.

Tabla 27

Clases del Paquete de Búsqueda de Patrones

Clase	Responsabilidad
`KnuthMorrisPratt`	Estructura `int[] pi` (función de prefijo).
`BoyerMoore`	Estructura `Tablas`: carácter malo (256 entradas) y sufijo bueno (m+1 entradas).

Clase	Responsabilidad
`RabinKarp`	Estructura `Hash`: hash del patrón y potencia h = d^(m−1) mod q. El módulo q es configurable.
`ResultadoBusqueda`	Posiciones encontradas y contadores: comparaciones de caracteres, alineamientos, coincidencias de hash y colisiones.
`Traza`	Registro opcional de cada alineamiento (posición s, comparaciones, evento, desplazamiento) para la interfaz y este documento. Si es `null` no se registra nada, para no alterar los tiempos medidos.
`MotorBusqueda`	Ejecuta los 3 algoritmos sobre el corpus, valida que coincidan y mide por separado preprocesamiento y búsqueda.

Nota. Elaboración propia.

Mayúsculas. Por defecto la búsqueda no distingue mayúsculas: patrón y texto se convierten a minúsculas **carácter por carácter** (`Character.toLowerCase`), lo que conserva la longitud del texto y permite que las posiciones encontradas sirvan sobre el abstract original. La interfaz tiene una casilla para distinguirlas.

6.2 Algoritmo Knuth-Morris-Pratt

Idea. El algoritmo ingenuo, al fallar después de coincidir q caracteres, retrocede en el texto y vuelve a comparar caracteres que ya leyó. KMP (Knuth et al., 1977) evita ese retroceso: esos q caracteres del texto son exactamente P[0..q−1], así que se sabe de antemano cuánto se puede desplazar el patrón sin perder ocurrencias. Ese dato es la **función de prefijo** (tabla de fallos):

```
pi[q] = longitud del prefijo propio más largo de P[0..q] que también es sufijo de P[0..q]   (su "borde")
```

Preprocesamiento (`preprocesar`). Se compara el patrón contra sí mismo. k es la longitud del borde actual; si P[k] ≠ P[q] el borde no se puede extender y se prueba con el borde del borde (k ← pi[k−1]):

```
pi[0] ← 0;  k ← 0
para q = 1 .. m−1:
    mientras k > 0 y P[k] ≠ P[q]:  k ← pi[k−1]
    si P[k] = P[q]:  k ← k + 1
    pi[q] ← k
```

Búsqueda (`buscar`). q cuenta cuántos caracteres del patrón coinciden con los últimos leídos; el patrón está alineado en s = i − q. Al fallar con q > 0 el patrón se desplaza q − pi[q−1] posiciones y se vuelve a comparar el mismo T[i]: el índice i del texto nunca retrocede.

```
q ← 0
para i = 0 .. n-1:
    mientras q > 0 y P[q] ≠ T[i]:  q ← pi[q-1]      (desplazamiento q - pi[q-1])
    si P[q] = T[i]:  q ← q + 1
    si q = m:  reportar ocurrencia en i - m + 1;  q ← pi[m-1]
```

6.3 Algoritmo Boyer-Moore

Idea. En Boyer, el patrón se compara de **derecha a izquierda**. Al fallar en la posición j del patrón con el carácter c = T[s+j], se calculan dos desplazamientos que no pueden saltarse una ocurrencia, y se aplica el mayor:

- **Regla del carácter malo:** alinear c con su última aparición en el patrón: `j - ultima(c)`. Si c no aparece en el patrón, `ultima(c) = -1` y el patrón salta completo por encima de c. Puede dar un valor ≤ 0 (si la última aparición de c está a la derecha de j); por eso no se usa sola.
- **Regla del sufijo bueno (versión fuerte)**: el sufijo P[j+1..m−1] ya coincidió con el texto. Se desplaza hasta la siguiente aparición de ese sufijo dentro del patrón precedida por un carácter distinto de P[j] (si estuviera precedida por P[j], volvería a fallar en el mismo sitio); si no existe, hasta el prefijo más largo del patrón que sea sufijo del tramo ya comparado.

Preprocesamiento.

1. `caracterMalo[c]` para los 256 códigos: se inicializa en −1 y se recorre el patrón de izquierda a derecha guardando `caracterMalo[P[j]] = j`; al terminar queda la **última**

aparición. Costo O(σ + m) con σ = 256. Los caracteres con código ≥ 256 (raros en los abstracts) se buscan recorriendo el patrón.

2. `sufijoBueno[0..m]` se construye en dos pasadas con el arreglo auxiliar `borde[i]`, que guarda dónde empieza el borde más ancho del sufijo P[i..m−1]:
    - *Caso 1:* recorriendo el patrón de derecha a izquierda, cada vez que un borde no se puede extender (P[i−1] ≠ P[j−1]) se registra `sufijoBueno[j] = j - i`: el sufijo que empieza en j aparece de nuevo en i precedido por otro carácter.
    - *Caso 2:* las posiciones que quedaron en 0 reciben el desplazamiento del borde más ancho de todo el patrón (`borde[0]`), es decir, se alinea el prefijo de P que coincide con parte del sufijo ya comparado.

Costo O(m) (cada posición de `borde` se recorre un número constante de veces en total).

Búsqueda.

```
s ← 0
mientras s ≤ n - m:
    j ← m - 1
    mientras j ≥ 0 y P[j] = T[s+j]:  j ← j - 1
    si j < 0: reportar ocurrencia en s;  s ← s + sufijoBueno[0]
    si no:    s ← s + max( sufijoBueno[j+1],  j - ultima(T[s+j]) )
```

6.4 Algoritmo Rabin-Karp

En Rabin-Karp, cada cadena de longitud m se interpreta como un número de m dígitos en base d = 256 (un dígito por carácter) y se reduce módulo un primo q:

$$hash(P) = (P[0]\cdot d^{(m-1)} + P[1]\cdot d^{(m-2)} + \ldots + P[m-1]) \bmod q$$

Solo se comparan caracteres en las ventanas del texto cuyo hash es igual al del patrón.

Preprocesamiento. hash(P) se calcula con la regla de Horner (hash ← (hash·d + P[i]) mod q, m pasos) y la potencia h = d^(m−1) mod q por multiplicaciones sucesivas (m−1 pasos). Reducir módulo q en cada paso evita el desbordamiento: el número completo tendría 8·m bits. Costo O(m).

Búsqueda (rolling hash):

El hash de la primera ventana se calcula con Horner y el de cada ventana siguiente se obtiene del anterior en O(1), quitando el carácter que sale (que pesa h) y agregando el que entra:

$$t(s+1) = (d \cdot (t(s) - T[s]\cdot h) + T[s+m]) \bmod q$$

Como la resta puede quedar negativa, la implementación suma q antes de reducir. Si t(s) = hash(P) la ventana se verifica carácter por carácter, porque dos cadenas distintas pueden tener el mismo hash (colisión espuria). Si se omitiera la verificación, el algoritmo reportaría falsos positivos.

Módulo: Por defecto q = 1 000 000 007 (primo grande: las colisiones son prácticamente imposibles). La interfaz permite elegir q = 101, un primo pequeño que produce colisiones visibles, útil para el caso de estudio.

6.5 Caso de Estudio

Patrón: generative (m = 10), elegido porque el dominio del proyecto es la IA generativa. Texto: última oración del abstract del artículo 16, "Generative Artificial Intelligence in Tertiary Education: Assessment Redesign Principles and Considerations" (Lye y Lim, 2024), en minúsculas, n = 222 caracteres:

```
0         10        20        30        40        50        60        70
it argues that more generative ai tools will emerge exponentially, and hence, en

80        90        100       110       120       130       140       150
gaging in an arms race against generative ai and policing the use of these techn

160       170       180       190       200       210       220
ologies may not address the fundamental issues in assessments.
```

Los tres algoritmos encuentran el patrón en las posiciones **20** y **111**. Todos los valores de esta sección fueron generados ejecutando el código real

(`app/src/test/java/.../GenerarCasoBusqueda.java`); en las trazas, el espacio se representa con

Para mayor comprensión sobre el patrón de búsqueda del algoritmo lo invitamos a visualizar el siguiente video, no olvide mejorarle la calidad del video a 1080p: Video 10 patron de busqueda del algoritmo Knuth-Morris-Pratt

6.5.1 KMP: Llenado de la Función de Prefijo y Traza

Llenado de pi para generative. preprocesar compara el patrón contra sí mismo. k es la longitud del borde que se está extendiendo; en cada posición q se compara P[k] con P[q]: si son iguales el borde crece (k ← k + 1), y si no, se retrocede al borde del borde (k ← pi[k − 1]) hasta que coincidan o k llegue a 0. Las trazas de esta sección se generaron con GenerarLlenadoTablas, que verifica que la tabla resultante sea idéntica a la que calcula el código.

Tabla 28

Llenado de la Función de Prefijo para el Patrón del Caso de Estudio

q	P[q]	k al entrar	Comparación P[k] vs P[q]	Acción	pi[q]
0	g	–	–	pi[0] = 0 por definición	**0**
1	e	0	P[0] = g ≠ e	k queda en 0	**0**
2	n	0	P[0] = g ≠ n	k queda en 0	**0**
3	e	0	P[0] = g ≠ e	k queda en 0	**0**
4	r	0	P[0] = g ≠ r	k queda en 0	**0**
5	a	0	P[0] = g ≠ a	k queda en 0	**0**
6	t	0	P[0] = g ≠ t	k queda en 0	**0**
7	i	0	P[0] = g ≠ i	k queda en 0	**0**
8	v	0	P[0] = g ≠ v	k queda en 0	**0**
9	e	0	P[0] = g ≠ e	k queda en 0	**0**

Nota. Elaboración propia.

Resultado: pi = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], con 9 comparaciones (la cota del preprocesamiento es 2m = 20). Todos los bordes son 0 porque ningún prefijo propio de "generative" es también sufijo: el único carácter con el que puede empezar un borde es "g", y "g" no vuelve a aparecer. Es un hallazgo representativo del lenguaje natural: en las palabras de los abstracts los bordes son raros (para intelligence solo pi[6] = 1, por la segunda "i"). Con pi = 0, cada fallo devuelve q a 0 y el patrón avanza hasta la posición del carácter que falló.

Para la visualización de como se construye el llenado la función de prefijo lo invitamos a visualizar el siguiente video recordando que debe poner la calidad a 1080p: Video 11 llenado de la función de prefijo

Llenado de pi para un patrón con bordes: `ababaca` (ejemplo de Cormen et al., cap. 32.4). Con "generative" el retroceso k ← pi[k − 1] nunca se ejecuta, así que se muestra también este patrón, donde sí aparece:

Tabla 29

Llenado de la Función de Prefijo para un Patrón con Retrocesos

q	P[q]	k al entrar	Comparaciones P[k] vs P[q]	Acción	pi[q]
0	a	–	–	pi[0] = 0 por definición	**0**
1	b	0	P[0] = a ≠ b	k queda en 0	**0**
2	a	0	P[0] = a = a	el borde crece: k = 1 ("a")	**1**
3	b	1	P[1] = b = b	el borde crece: k = 2 ("ab")	**2**
4	a	2	P[2] = a = a	el borde crece: k = 3 ("aba")	**3**
5	c	3	P[3] = b ≠ c; P[1] = b ≠ c; P[0] = a ≠ c	k = pi[2] = 1; k = pi[0] = 0; k queda en 0	**0**
6	a	0	P[0] = a = a	el borde crece: k = 1	**1**

Nota. Elaboración propia.

En q = 5 se ve el mecanismo central de KMP. El borde "aba" no se puede extender con "c". En lugar de empezar de cero, se prueba el borde del borde: pi[2] = 1, es decir "a". Tampoco se extiende con "c", así que se pasa a pi[0] = 0 y finalmente k = 0. Resultado: pi = [0, 0, 1, 2, 3, 0, 1], con 8 comparaciones (cota 2m = 14). k aumenta como mucho 1 por posición y cada retroceso lo disminuye, así que el total de comparaciones es O(m)

Traza de la búsqueda. KMP hizo 194 alineamientos y 221 comparaciones. Se muestran los 10 pasos con más de una comparación. Los otros 184 son fallos inmediatos en j = 0, con desplazamiento +1:

Tabla 30

Traza de KMP sobre el Texto del Caso de Estudio

Paso	s	Texto T[s..s+9]	Comp.	Evento	Desplaz.
6	5	`gues_that_`	2	fallo en j=1 ('u' vs 'e'), q = pi[0] = 0	+1
21	20	`generative`	10	**ocurrencia**	+10
41	49	`ge_exponen`	3	fallo en j=2 (' ' vs 'n'), q = pi[1] = 0	+2
71	80	`gaging_in_`	2	fallo en j=1 ('a' vs 'e'), q = pi[0] = 0	+1
73	82	`ging_in_an`	2	fallo en j=1 ('i' vs 'e'), q = pi[0] = 0	+1
76	85	`g_in_an_ar`	2	fallo en j=1 (' ' vs 'e'), q = pi[0] = 0	+1
95	104	`gainst_gen`	2	fallo en j=1 ('a' vs 'e'), q = pi[0] = 0	+1
102	111	`generative`	10	**ocurrencia**	+10
118	136	`g_the_use_`	2	fallo en j=1 (' ' vs 'e'), q = pi[0] = 0	+1
145	163	`gies_may_n`	2	fallo en j=1 ('i' vs 'e'), q = pi[0] = 0	+1

Nota. Elaboración propia.

Ejemplo: con el patrón en s = 49 coinciden "g" (T[49]) y "e" (T[50]), q = 2, y falla T[51] = ' ' contra P[2] = 'n'. KMP no retrocede a T[50]: hace q ← pi[1] = 0, el patrón avanza q −

pi[q−1] = 2 posiciones (s = 51) y compara el mismo T[51] contra P[0]. Por eso las comparaciones (221) no superan 2n = 444: cada carácter del texto se compara una vez cuando avanza i, más una vez por cada retroceso de q.

6.5.2 Boyer-Moore: Llenado de las Tablas Heurísticas y Traza

Llenado de la tabla de carácter malo. Se inicializan las 256 entradas en −1 y se recorre el patrón de izquierda a derecha asignando caracterMalo[P[j]] = j. Como cada asignación sobrescribe la anterior, al final queda la última aparición de cada carácter:

Tabla 31

Llenado de la Tabla de Carácter Malo

j	P[j]	Asignación	Estado de la tabla (solo letras del patrón; el resto vale −1)
0	g	caracterMalo['g'] = 0	g=0
1	e	caracterMalo['e'] = 1	g=0 e=1
2	n	caracterMalo['n'] = 2	g=0 e=1 n=2
3	e	caracterMalo['e'] = 3 (**sobrescribe 1**)	g=0 e=3 n=2
4	r	caracterMalo['r'] = 4	g=0 e=3 n=2 r=4
5	a	caracterMalo['a'] = 5	g=0 e=3 n=2 r=4 a=5
6	t	caracterMalo['t'] = 6	g=0 e=3 n=2 r=4 a=5 t=6
7	i	caracterMalo['i'] = 7	g=0 e=3 n=2 r=4 a=5 t=6 i=7
8	v	caracterMalo['v'] = 8	g=0 e=3 n=2 r=4 a=5 t=6 i=7 v=8
9	e	caracterMalo['e'] = 9 (**sobrescribe 3**)	g=0 e=9 n=2 r=4 a=5 t=6 i=7 v=8

Nota. Elaboración propia.

Tabla 32

Tabla de Carácter Malo Final

c	g	e	n	r	a	t	i	v	otro
ultima(c)	0	9	2	4	5	6	7	8	−1

Nota. Elaboración propia.

Por qué importa guardar la última aparición: si se guardara la primera ("e" = 1), al fallar contra una "e" del texto el patrón saltaría hasta alinear la "e" de la posición 1 y se podría pasar por encima de una ocurrencia que alinea la "e" de la posición 3 o la 9. Con la última se obtiene siempre el desplazamiento seguro más corto.

Llenado de la tabla de sufijo bueno. sufijoBueno[j] es el desplazamiento cuando el sufijo P[j..m−1] ya coincidió y el fallo ocurre en j − 1. Se construye en dos pasadas con el arreglo auxiliar borde[i] (posición donde empieza el borde más ancho del sufijo P[i..m−1]; m + 1 si no tiene).

Caso 1, de derecha a izquierda. Para cada i se intenta extender el borde del sufijo comparando P[i−1] con P[j−1]. Si no se puede, ese sufijo de longitud m − j aparece otra vez en el patrón precedido por un carácter distinto, y se registra el desplazamiento j − i:

Tabla 33

Cálculo de los Bordes para la Regla del Sufijo Bueno

i	j	Comparación P[i−1] vs P[j−1]	Acción	borde
10	11	–	inicio: el sufijo vacío no tiene borde	borde[10] = 11
10→9	11→10	j = 11 > m, nada que comparar	avanza	borde[9] = 10

i	j	Comparación P[i−1] vs P[j−1]	Acción	borde
9→8	10→10	P[8] = v ≠ P[9] = e	**sufijoBueno[10] = 10 − 9 = 1**; j = borde[10] = 11	borde[8] = 10
8→7 … 5→4	10	P[7] = i, P[6] = t, P[5] = a, P[4] = r, cada una ≠ e	sufijoBueno[10] ya tenía 1, no cambia	borde[7..4] = 10
4→3	10→9	P[3] = e = P[9] = e	el borde se extiende	borde[3] = 9
3→2	9→10	P[2] = n ≠ P[8] = v; luego P[2] = n ≠ P[9] = e	**sufijoBueno[9] = 9 − 3 = 6**; j = borde[9] = 10; sufijoBueno[10] ya tenía 1	borde[2] = 10
2→1	10→9	P[1] = e = P[9] = e	el borde se extiende	borde[1] = 9
1→0	9→10	P[0] = g ≠ P[8] = v; luego P[0] = g ≠ P[9] = e	sufijoBueno[9] ya tenía 6, no cambia	borde[0] = 10

Nota. Elaboración propia.

Tras el caso 1: sufijoBueno = [0, 0, 0, 0, 0, 0, 0, 0, 0, **6**, **1**]. El valor 6 se registró en i = 3: el sufijo "e" (j = 9) reaparece en la posición 3 precedido por "n", que es distinto de la "v" que precede a la "e" final.

Caso 2. Las posiciones que quedaron en 0 no tienen otra aparición de su sufijo. Reciben el desplazamiento del borde más ancho de todo el patrón, j = borde[0] = 10. Como "generative" no tiene borde, ese desplazamiento es m = 10: se puede saltar el patrón completo. Las posiciones 9 y 10 conservan su valor del caso 1:

Tabla 34

Tabla de Sufijo Bueno después del Caso 2

i	0	1	2	3	4	5	6	7	8	9	10
antes (caso 1)	0	0	0	0	0	0	0	0	0	6	1
después (caso 2)	10	10	10	10	10	10	10	10	10	6	1

Nota. Elaboración propia.

Con ababaca, que sí tiene borde ("a"), el caso 2 asigna 6 = m − 1 en lugar de m. El resultado es sufijoBueno = [6, 6, 6, 6, 6, 6, 2, 1]: el patrón no puede saltar completo, porque su "a" final podría ser el inicio de otra ocurrencia.

Tablas finales y su significado (sufijoBueno[j+1] es el desplazamiento cuando falla en j; sufijoBueno[0], tras una ocurrencia completa):

Tabla 35

Tablas Finales de Sufijo Bueno

i	0	1	2	3	4	5	6	7	8	9	10
P[i]	g	e	n	e	r	a	t	i	v	e	–
borde[i]	10	9	10	9	10	10	10	10	10	10	11
sufijoBueno[i]	10	10	10	10	10	10	10	10	10	6	1

Nota. Elaboración propia.

- sufijoBueno[10] = 1: si falla en el primer carácter comparado (j = 9) todavía no hay sufijo bueno; esta regla solo garantiza avanzar 1, y es la de carácter malo la que da el salto.
- sufijoBueno[9] = 6: si coincidió el sufijo "e" y falla en j = 8, la siguiente "e" del patrón precedida por un carácter distinto de P[8] = 'v' está en la posición 3 (precedida por 'n'). Desplazar 9 − 3 = 6 la alinea con la "e" del texto. La "e" de la posición 1 también sirve, pero 3 está más a la derecha y da el desplazamiento seguro más corto.
- El resto vale 10 = m: ningún sufijo más largo ("ve", "ive", …) se repite en el patrón y el patrón no tiene borde, así que se puede saltar completo.

Traza completa. Boyer-Moore hizo solo 28 alineamientos y 49 comparaciones:

Tabla 36

Traza de Boyer-Moore sobre el Texto del Caso de Estudio

Paso	s	Texto T[s..s+9]	Comp.	Evento (carácter malo / sufijo bueno)	Desplaz.
1	0	`it_argues_`	1	fallo en j=9 (' ' vs 'e'); CM = 9−(−1) = 10, SB = 1	+10
2	10	`that_more_`	1	fallo en j=9 (' ' vs 'e'); CM = 10, SB = 1	+10
3	20	`generative`	10	**ocurrencia**; SB[0] = 10	+10
4	30	`_ai_tools_`	1	fallo en j=9 (' ' vs 'e'); CM = 10, SB = 1	+10
5	40	`will_emerg`	1	fallo en j=9 ('g' vs 'e'); CM = 9−0 = 9, SB = 1	+9
6	49	`ge_exponen`	1	fallo en j=9 ('n' vs 'e'); CM = 9−2 = 7, SB = 1	+7
7	56	`nentially,`	1	fallo en j=9 (',' vs 'e'); CM = 10, SB = 1	+10
8	66	`_and_hence`	2	fallo en j=8 ('c' vs 'v'); CM = 8−(−1) = 9, SB = 6	+9
9	75	`e,_engagin`	1	fallo en j=9 ('n' vs 'e'); CM = 7, SB = 1	+7
10	82	`ging_in_an`	1	fallo en j=9 ('n' vs 'e'); CM = 7, SB = 1	+7
11	89	`_an_arms_r`	1	fallo en j=9 ('r' vs 'e'); CM = 9−4 = 5, SB = 1	+5
12	94	`rms_race_a`	1	fallo en j=9 ('a' vs 'e'); CM = 9−5 = 4, SB = 1	+4
13	98	`race_again`	1	fallo en j=9 ('n' vs 'e'); CM = 7, SB = 1	+7
14	105	`ainst_gene`	2	fallo en j=8 ('n' vs 'v'); CM = 8−2 = 6, SB = 6	+6
15	111	`generative`	10	**ocurrencia**; SB[0] = 10	+10
16	121	`_ai_and_po`	1	fallo en j=9 ('o' vs 'e'); CM = 10, SB = 1	+10
17	131	`licing_the`	2	fallo en j=8 ('h' vs 'v'); CM = 9, SB = 6	+9
18	140	`e_use_of_t`	1	fallo en j=9 ('t' vs 'e'); CM = 9−6 = 3, SB = 1	+3
19	143	`se_of_thes`	1	fallo en j=9 ('s' vs 'e'); CM = 10, SB = 1	+10
20	153	`e_technolo`	1	fallo en j=9 ('o' vs 'e'); CM = 10, SB = 1	+10
21	163	`gies_may_n`	1	fallo en j=9 ('n' vs 'e'); CM = 7, SB = 1	+7
22	170	`y_not_addr`	1	fallo en j=9 ('r' vs 'e'); CM = 5, SB = 1	+5
23	175	`_address_t`	1	fallo en j=9 ('t' vs 'e'); CM = 3, SB = 1	+3
24	178	`dress_the_`	1	fallo en j=9 (' ' vs 'e'); CM = 10, SB = 1	+10
25	188	`fundamenta`	1	fallo en j=9 ('a' vs 'e'); CM = 4, SB = 1	+4
26	192	`amental_is`	1	fallo en j=9 ('s' vs 'e'); CM = 10, SB = 1	+10
27	202	`sues_in_as`	1	fallo en j=9 ('s' vs 'e'); CM = 10, SB = 1	+10
28	212	`sessments.`	1	fallo en j=9 ('.' vs 'e'); CM = 10, SB = 1	+10

Nota. Elaboración propia.

Representación gráfica de los primeros desplazamientos (el patrón se compara de derecha a izquierda, empezando por el carácter alineado con su última letra):

```
texto:  it argues that more generative ai tools will emerge exponentially, and hence, engaging in ...
s=0     generative                                                falla en j=9 con ' ': CM=10    -> +10
s=10              generative                                      falla en j=9 con ' ': CM=10    -> +10
s=20                       generative                             ocurrencia                   -> +10
s=30                                generative                    falla en j=9 con ' ': CM=10    -> +10
s=40                                         generative           falla en j=9 con 'g': CM=9     -> +9
s=49                                                  generative  falla en j=9 con 'n': CM=7     -> +7
s=56                                                        generative falla en j=9 con ',': CM=10    -> +10
s=66                                                                 generative  falla en j=8 con 'c': max(9,6) -> +9
```

Pasos destacados:

- **Paso 5 (s = 40):** el último carácter de la ventana es T[49] = 'g'. La última "g" del patrón está en la posición 0, así que el carácter malo desplaza 9 − 0 = 9 para que esa "g" quede bajo T[49]. Ahí empieza "ge exp…", que coincide en sus 2 primeros caracteres pero no es ocurrencia (paso 6).
- **Paso 8 (s = 66):** T[75] = 'e' coincide con P[9]; falla en j = 8 con 'c'. Carácter malo: 'c' no está en el patrón, 8 − (−1) = 9. Sufijo bueno: `sufijoBueno[9] = 6`. Se aplica el mayor, **+9**.
- **Paso 14 (s = 105):** coincide "e" y falla en j = 8 con 'n'. Carácter malo: la última 'n' del patrón está en 2, 8 − 2 = 6. Sufijo bueno: también 6. El desplazamiento de 6 deja el patrón en s = 111, donde está la segunda ocurrencia.

6.5.3 Rabin-Karp: Cálculo Iterativo del Hash y Colisiones

Con q = 101 (primo pequeño), d = 256:

Hash del patrón por la regla de Horner:

Tabla 37

Hash del Patrón por la Regla de Horner

i	P[i] (código)	Cálculo	Resultado
0	g (103)	(0·256 + 103) mod 101	2
1	e (101)	(2·256 + 101) mod 101	7
2	n (110)	(7·256 + 110) mod 101	84
3	e (101)	(84·256 + 101) mod 101	92
4	r (114)	(92·256 + 114) mod 101	32
5	a (97)	(32·256 + 97) mod 101	7
6	t (116)	(7·256 + 116) mod 101	90
7	i (105)	(90·256 + 105) mod 101	16
8	v (118)	(16·256 + 118) mod 101	73
9	e (101)	(73·256 + 101) mod 101	**3**

Nota. Elaboración propia.

$$\text{hash}(P) = 3 \quad y \quad h = 256^9 \bmod 101 = 24.$$

Rolling hash. La primera ventana, "it argues " (T[0..9]), tiene hash t(0) = 100 (Horner sobre T[0..9]). Para pasar a la ventana s = 1 sale T[0] = 'i' (105) y entra T[10] = 't' (116):

$$\begin{aligned} T[0]\cdot h \bmod q &= 105 \cdot 24 \bmod 101 = 2520 \bmod 101 = 96 \\ t(0) - 96 &= 100 - 96 = 4 \\ t(1) &= (4 \cdot 256 + 116) \bmod 101 = 1140 \bmod 101 = 29 \end{aligned}$$

que coincide con el hash calculado por el programa para "t argues t". El cálculo se repite en O(1) para cada una de las 213 ventanas:

Tabla 38

Hash de las Ventanas del Texto en Rabin-Karp

s	Ventana	Hash t(s)	Resultado
0	`it_argues_`	100	≠ 3, siguiente ventana
1	`t_argues_t`	29	≠ 3
2	`_argues_th`	6	≠ 3
3	`argues_tha`	56	≠ 3
4	`rgues_that`	42	≠ 3
5	`gues_that_`	97	≠ 3
…	…	…	…
20	`generative`	3	= hash(P) → verificación de 10 caracteres: **ocurrencia**
111	`generative`	3	= hash(P) → verificación: **ocurrencia**
122	`ai_and_pol`	3	= hash(P) → verificación: falla en j = 0 ('a' vs 'g'): **colisión espuria**

Nota. Elaboración propia.

Con q = 101 hubo 3 coincidencias de hash: 2 ocurrencias reales y 1 colisión. La colisión solo costó una comparación, porque la verificación se detiene en el primer carácter distinto. En total se hicieron 21 comparaciones de caracteres (10 + 10 + 1).

Con q = 1 000 000 007 el mismo párrafo da hash(P) = 89 521 723, h = 80 065 005, 2 coincidencias de hash, 0 colisiones y 20 comparaciones. Un módulo grande reduce la probabilidad de colisión a ≈ 1/q por ventana.

6.5.4 Resumen del Caso de Estudio

Tabla 39

Resumen del Caso de Estudio de Búsqueda de Patrones

Algoritmo	Alineamientos / ventanas	Comparaciones de caracteres	Comparaciones / n
KMP	194	221	0.995
Boyer-Moore	28	49	0.221
Rabin-Karp (q = 101)	213 (1 colisión)	21	0.095

Nota. Elaboración propia.

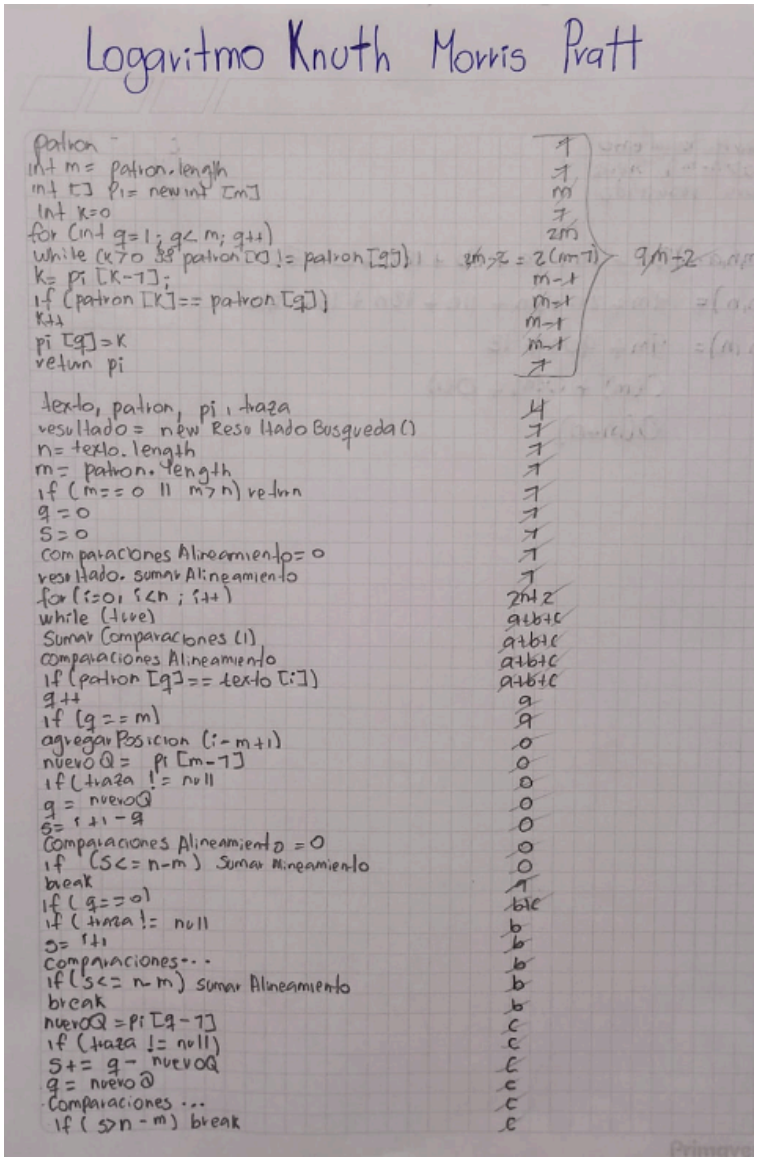
6.6 Análisis Crítico: Complejidad Teórica frente a Rendimiento Observado

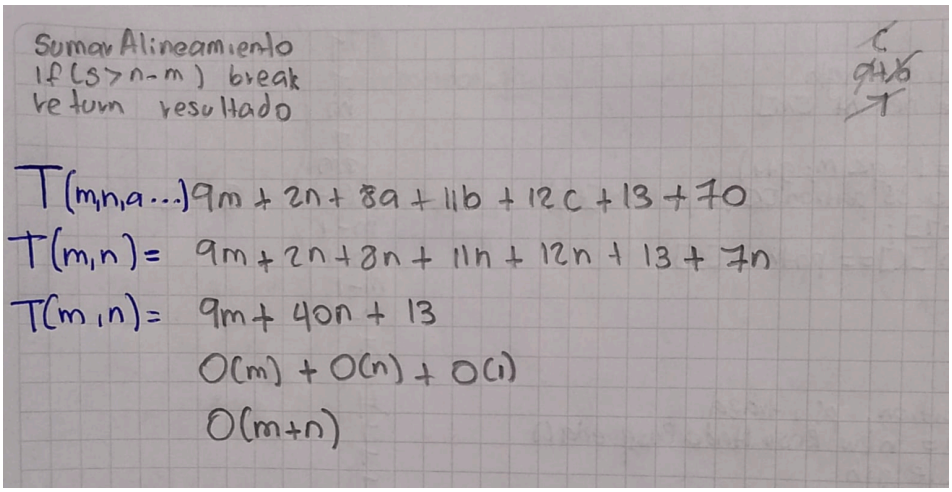
6.6.1 Complejidad Asintótica

- **KMP — preprocesamiento O(m), búsqueda O(n).** Hay ciclos anidados , pero el análisis es amortizado: en la búsqueda, q aumenta como máximo 1 por cada carácter leído (n veces en total) y cada iteración del mientras lo disminuye al menos 1. Como $q \geq 0$, el mientras no puede ejecutarse en total más de n veces. $T(n) \leq c \cdot (n + n) = 2c \cdot n \Rightarrow O(n)$, independiente del contenido del texto. El mismo argumento con k da O(m) para el preprocesamiento. Total O(n + m) siempre, sin peor caso cuadrático.

Figura 14

Tiempo de ejecución y complejidad asintótica

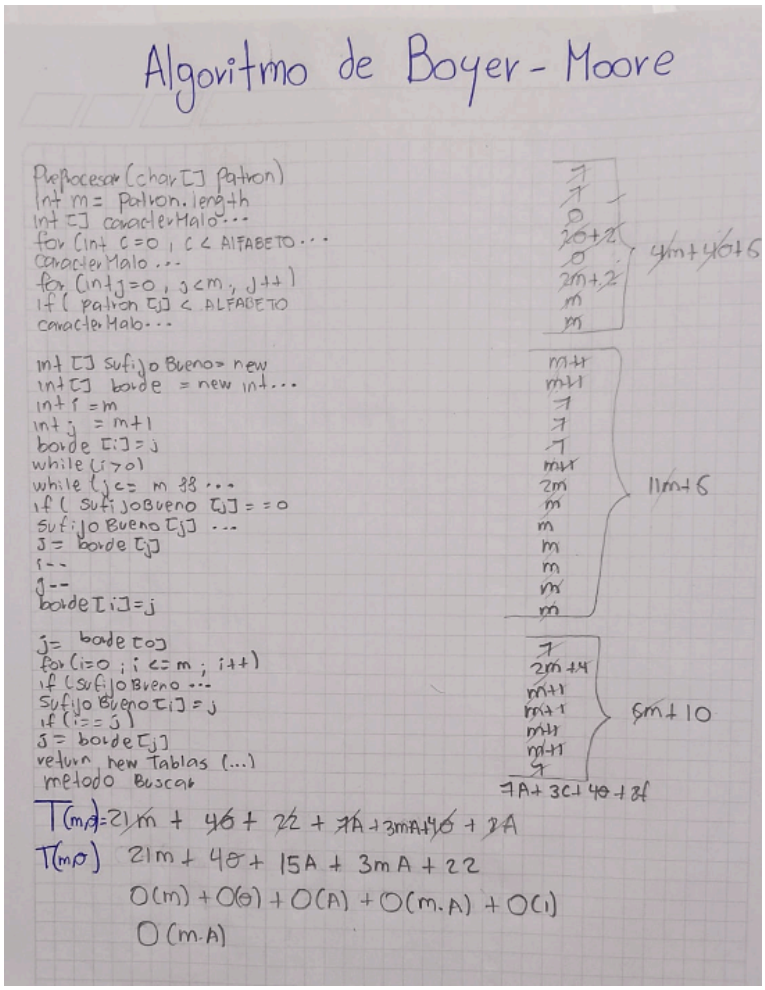




- **Boyer-Moore — preprocesamiento O(m + σ), búsqueda O(n·m) peor caso.** El preprocesamiento recorre las σ = 256 entradas de la tabla de carácter malo más dos pasadas lineales sobre el patrón. En la búsqueda, en el peor caso (texto aaaa…a, patrón aa…a) cada uno de los n − m + 1 alineamientos compara m caracteres: (n−m+1)·m ⇒ O(n·m). En el mejor caso, cada alineamiento falla en la primera comparación con un carácter que no está en el patrón y salta m posiciones: ⌈n/m⌉ comparaciones ⇒ O(n/m), sublineal. Es el único de los tres que puede no leer todos los caracteres del texto.

Figura 15

Tiempo de ejecución y complejidad asintótica



- **Rabin-Karp — preprocesamiento O(m), búsqueda O(n + m) esperado, O(n·m) peor caso.** Hay n − m + 1 ventanas de costo O(1) (rolling hash) más O(m) por cada coincidencia de hash. Si hay c ocurrencias y el hash se comporta como uniforme, se esperan ≈ (n−m+1)/q colisiones: T = O(n + m·(c + n/q)), que con q grande es O(n + m).

En el peor caso (q muy pequeño o texto adversario, casi todas las ventanas colisionan) se verifican m caracteres en cada ventana: O(n·m).

Figura 16

Tiempo de ejecución y complejidad asintótica

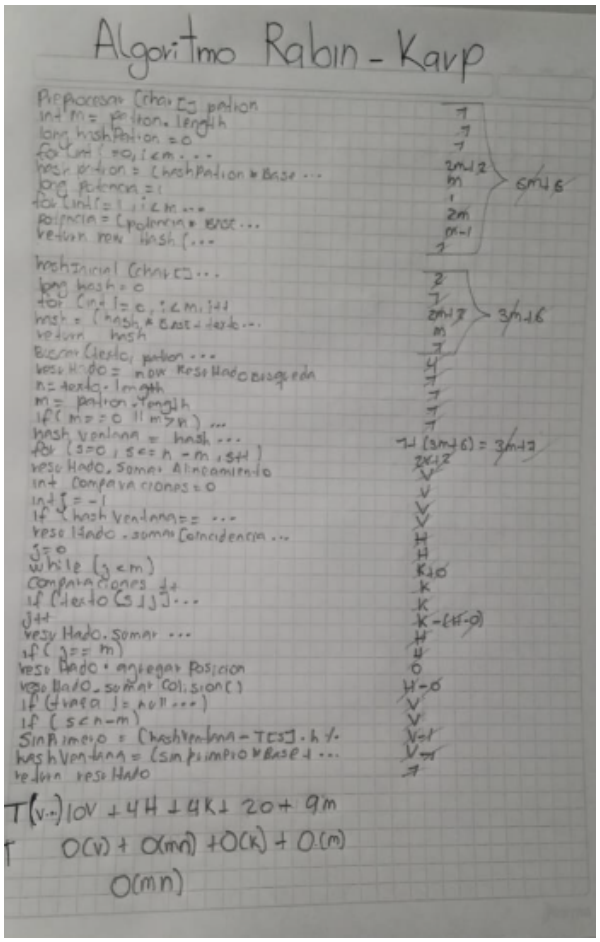


Tabla 40

Complejidad Asintótica de los Algoritmos de Búsqueda

Algoritmo	Preprocesamiento	Búsqueda (peor caso)	Búsqueda (caso típico)	Espacio extra
KMP	O(m)	O(n)	Θ(n)	O(m)
Boyer-Moore	O(m + σ)	O(n·m)	~O(n/m)	O(m + σ)
Rabin-Karp	O(m)	O(n·m)	O(n + m)	O(1)

Nota. Elaboración propia.

6.6.2 Medición: Preprocesamiento frente a Búsqueda

MotorBusqueda (y GenerarCasoBusqueda para estas tablas) mide con System.nanoTime(): primero un calentamiento de la JVM sin medir (para que el compilador JIT optimice el código), luego varias repeticiones (201 en estas tablas), y se reporta la mediana en milisegundos, que no se ve afectada por pausas aisladas del recolector de basura. El preprocesamiento se mide una vez por patrón; la búsqueda, sobre los 20 abstracts completos (n = 28 041 caracteres en minúsculas):

Tabla 41

Tiempos Medidos de Preprocesamiento y de Búsqueda en el Corpus

Patrón	m	Algoritmo	Preproc. (ms)	Búsqueda (ms)	Comparaciones	Comp. / carácter	Alineamientos
ai	2	KMP	0.0002	0.1286	29 878	1.066	27 852
ai	2	Boyer-Moore	0.0016	0.1921	15 621	0.557	14 522
ai	2	Rabin-Karp	0.0002	0.6235	338	0.012	28 021
data	4	KMP	0.0002	0.1229	28 976	1.033	27 942
data	4	Boyer-Moore	0.0016	0.0948	8 132	0.290	7 559
data	4	Rabin-Karp	0.0004	0.6202	32	0.001	27 981

Patrón	m	Algoritmo	Preproc. (ms)	Búsqueda (ms)	Comparaciones	Comp. / carácter	Alineamientos
learning	8	KMP	0.0001	0.1218	28 834	1.028	27 454
learning	8	Boyer-Moore	0.0004	0.0616	4 941	0.176	4 352
learning	8	Rabin-Karp	0.0012	0.7109	320	0.011	27 901
generative	10	KMP	0.0002	0.1196	28 397	1.013	27 593
generative	10	Boyer-Moore	0.0003	0.0448	4 300	0.153	3 655
generative	10	Rabin-Karp	0.0004	0.6864	120	0.004	27 861
artificial intelligence	23	KMP	0.0003	0.1198	29 570	1.055	26 898
artificial intelligence	23	Boyer-Moore	0.0004	0.0249	2 632	0.094	1 942
artificial intelligence	23	Rabin-Karp	0.0006	0.6862	506	0.018	27 601
the use of generative artificial intelligence	45	KMP	0.0006	0.1487	29 090	1.037	26 273
the use of generative artificial intelligence	45	Boyer-Moore	0.0006	0.0168	1 849	0.066	1 296
the use of generative artificial intelligence	45	Rabin-Karp	0.0013	0.6987	45	0.002	27 161

Nota. Elaboración propia.

Lectura de la tabla.

1. **El preprocesamiento es despreciable frente a la búsqueda.** Cuesta entre 0.0001 y 0.0016 ms, entre 25 y más de 1000 veces menos que la búsqueda, porque depende de m (2 a 45) y la búsqueda depende de n (28 041). Además se hace una sola vez por patrón y se reutiliza en los 20 abstracts: su costo se amortiza. Solo dominaría con textos muy

cortos o con un patrón distinto para cada texto. Estos valores están cerca de la resolución del reloj (System.nanoTime, del orden de 0.0001 ms), así que de ellos solo es confiable el orden de magnitud.

2. **El término σ de Boyer-Moore se refleja en las mediciones.** Su preprocesamiento es el mayor o igual al de KMP en todos los patrones, y con m = 2 y m = 4 llega a ser unas 8 veces el de KMP (0.0016 frente a 0.0002 ms): con un patrón tan corto, inicializar las 256 entradas de la tabla de carácter malo cuesta más que recorrer el patrón. Es el O(m + σ) de la teoría, en el que domina σ cuando m es pequeño.
3. **KMP es lineal e insensible a m.** Hace ≈ 1.01–1.07 comparaciones por carácter con cualquier patrón, por debajo de la cota teórica de 2n. Su tiempo de búsqueda es prácticamente constante (≈ 0.12 ms) para m de 2 a 45.
4. **Boyer-Moore es sublineal y mejora con m.** Las comparaciones por carácter bajan de 0.557 (m = 2) a 0.066 (m = 45), aproximadamente proporcional a 1/m, como predice el caso típico O(n/m). Su tiempo baja de 0.19 a 0.017 ms. Con m = 2 es más lento que KMP: los saltos son de 2 como máximo y el cálculo de los dos desplazamientos tiene más costo por paso que la comparación simple de KMP.
5. **Rabin-Karp casi no compara caracteres, pero es el más lento.** Solo compara en las verificaciones (0.001 a 0.018 comparaciones por carácter), pero procesa **todas** las n − m + 1 ventanas, y cada actualización del rolling hash cuesta dos multiplicaciones y dos operaciones módulo (`%`) sobre enteros de 64 bits. La división entera es de las instrucciones más costosas del procesador, mucho más que comparar dos caracteres. Por

eso es ≈ 5–6 veces más lento que KMP aunque ambos son O(n). La notación O oculta esta constante y solo la medición la revela.

6.6.3 Crecimiento con n

Para comprobar empíricamente el orden de crecimiento se buscó `generative` en el corpus concatenado k veces (k = 1, 2, 4, 8, 16):

Tabla 42

Crecimiento del Tiempo de Búsqueda con el Tamaño del Texto

k	n	KMP (ms)	Boyer-Moore (ms)	Rabin-Karp (ms)	Comp./carácter KMP	Comp./carácter BM	Comp./carácter RK
1	28 061	0.1186	0.0479	0.6806	1.019	0.154	0.004
2	56 122	0.2392	0.0884	1.3548	1.019	0.154	0.004
4	112 244	0.4660	0.1707	2.7085	1.019	0.154	0.004
8	224 488	0.9433	0.3411	5.5792	1.019	0.154	0.004
16	448 976	2.0946	0.7200	11.1237	1.019	0.154	0.004

Nota. Elaboración propia.

Al duplicar n, el tiempo de los tres algoritmos se duplica (KMP: 0.12 → 0.24 → 0.47 → 0.94 → 2.09 ms) y las comparaciones por carácter se mantienen constantes. Ese es el comportamiento lineal que predice la teoría. Sobre los abstracts ninguno muestra el crecimiento O(n·m) de su peor caso: el texto natural no tiene la estructura repetitiva que lo provoca. La sección siguiente lo provoca a propósito.

6.6.4 Peor Caso: la Diferencia entre O(n) y O(n·m), Medida

Para contrastar la cota de peor caso, se construyó la entrada que la teoría señala como la más desfavorable: un texto de n = 100 000 letras "a" y un patrón de m letras "a". El patrón aparece en todas las posiciones (n − m + 1 ocurrencias, muchas solapadas) (`GenerarLlenadoTablas`, mediana de 51 repeticiones tras calentamiento):

Tabla 43

Comparaciones y Tiempos con un Texto y un Patrón Formados Solo por la Letra a

m	(n − m + 1)·m	KMP: comparaciones	Boyer-Moore: comparaciones	Rabin-Karp: comparaciones	KMP (ms)	Boyer-Moore (ms)	Rabin-Karp (ms)
5	499 980	100 000	499 980	499 980	2.450	2.995	3.566
10	999 910	100 000	999 910	999 910	2.115	2.912	3.837
20	1 999 620	100 000	1 999 620	1 999 620	1.562	3.686	4.552
50	4 997 550	100 000	4 997 550	4 997 550	1.477	7.917	7.883
100	9 990 100	100 000	9 990 100	9 990 100	1.327	**11.859**	**14.102**

Nota. Elaboración propia.

La medición confirma exactamente la teoría:

- **KMP hace siempre n = 100 000 comparaciones**, sea cual sea m. Cada carácter coincide, q avanza y, tras cada ocurrencia, pi[m − 1] = m − 1 conserva todo lo ya comparado, así que el texto se lee una sola vez. Su tiempo (1.3–2.5 ms) no crece con m: es la garantía O(n) del análisis amortizado.
- **Boyer-Moore hace exactamente (n − m + 1)·m comparaciones**: el valor teórico de su peor caso, O(n·m). En cada uno de los n − m + 1 alineamientos compara los m caracteres

de derecha a izquierda, encuentra una ocurrencia y su sufijo bueno (`sufijoBueno[0]` = 1, porque el borde de "aa…a" es "aa…a" sin una letra) solo le permite avanzar 1 posición. Su tiempo crece con m: de 3.0 ms (m = 5) a 11.9 ms (m = 100).

- **Rabin-Karp también hace (n − m + 1)·m comparaciones**: todas las ventanas tienen el mismo hash que el patrón (son la misma cadena), así que todas se verifican carácter por carácter. Es su peor caso O(n·m), y su tiempo crece igual: de 3.6 a 14.1 ms.

En texto natural Boyer-Moore era el más rápido y KMP más lento que él. En esta entrada, con m = 100, KMP es 9 veces más rápido que Boyer-Moore y 10 veces más rápido que Rabin-Karp. La notación O describe ese cambio: los tres tienen búsqueda O(n) en el caso típico, pero solo KMP lo garantiza en el peor caso.

Como contraste, con el patrón "b" + "aa…a", que **no** aparece en el texto, los papeles se invierten:

Tabla 44

Comparaciones y Tiempos con un Patrón que no Aparece en el Texto

m	KMP: comparaciones	Boyer-Moore: comparaciones	Rabin-Karp: comparaciones	KMP (ms)	Boyer-Moore (ms)	Rabin-Karp (ms)
10	99 991	100 000	0	0.667	0.358	1.946
100	99 901	100 000	0	0.656	**0.098**	2.027

Nota. Elaboración propia.

Boyer-Moore compara de derecha a izquierda: las m − 1 "a" coinciden y falla en la "b". Su sufijo bueno ("aa…a" no precedido por "b") le permite saltar m posiciones, así que hace ≈ n/m alineamientos de m comparaciones cada uno. El total es n, pero con m = 100 hace solo ~1

000 alineamientos y por eso tarda 0.098 ms. Rabin-Karp no compara ningún carácter, porque ningún hash coincide, pero sigue pagando las n − m + 1 actualizaciones del rolling hash (≈ 2 ms). KMP vuelve a hacer ≈ n comparaciones.

6.6.5 Colisiones de Rabin-Karp según el Módulo

Sobre el corpus completo:

Tabla 45

Colisiones de Rabin-Karp según el Módulo

q	Patrón	Coincidencias de hash	Colisiones espurias	Ocurrencias reales	Búsqueda (ms)
101	ai	231	62	169	0.7039
101	generative	303	291	12	0.7062
1 009	ai	181	12	169	0.6944
1 009	generative	26	14	12	0.7044
1 000 000 007	ai	169	0	169	0.6230
1 000 000 007	generative	12	0	12	0.6816

Nota. Elaboración propia.

Con q = 101 y `generative` hay 291 colisiones, muy cerca de las (n − m + 1)/q = 27 861/101 ≈ 276 que se esperan si el hash reparte las ventanas de forma uniforme. Con q = 1 000 000 007 desaparecen. Para `ai` (m = 2) hay menos colisiones de las esperadas porque los pares de letras del inglés no son uniformes. El tiempo casi no cambia con las colisiones: cada verificación espuria suele terminar en el primer carácter, así que su costo real es O(1) y no O(m). El peor caso O(n·m) requiere que casi todas las ventanas tengan el mismo hash que el patrón y se verifiquen completas, algo que no ocurre en texto natural pero sí en la entrada repetitiva.

6.6.6 Conclusión

Tabla 46

Comparación Final de los Tres Algoritmos de Búsqueda

Criterio	KMP	Boyer-Moore	Rabin-Karp
Garantía de peor caso	**O(n) siempre**	O(n·m)	O(n·m)
Peor caso medido (n = 100 000, m = 100, sección 6.6.4)	**100 000 comparaciones, 1.3 ms**	9 990 100 comparaciones, 11.9 ms	9 990 100 comparaciones, 14.1 ms
Rendimiento observado en los abstracts	≈ 0.12 ms, constante	**0.02–0.19 ms, mejora con m**	≈ 0.62–0.71 ms
Comparaciones por carácter	≈ 1	**0.07–0.56**	≈ 0.01 (pero procesa todas las ventanas)
Preprocesamiento	O(m), mínimo	O(m + σ), notable con m pequeño	O(m), mínimo

Nota. Elaboración propia.

- **Boyer-Moore** es el más rápido para buscar palabras y frases en los abstracts: el alfabeto del inglés es amplio y la mayoría de los fallos ocurre en el primer carácter comparado, que permite saltos de casi m posiciones.
- **KMP** es la opción más predecible: no depende del contenido del texto, nunca retrocede en él (sirve para leer un flujo de datos de una sola pasada) y no tiene peor caso cuadrático. En texto natural su ventaja sobre el algoritmo ingenuo es pequeña, porque las palabras casi no tienen bordes (pi = 0, sección 6.5.1).
- **Rabin-Karp** compara menos caracteres pero paga la aritmética modular en cada ventana. Su ventaja real aparece en problemas que este requerimiento no exige: buscar **muchos**

patrones de la misma longitud a la vez (se comparan los hashes contra un conjunto) o detectar fragmentos repetidos entre documentos (plagio).

El análisis asintótico explica lo observado. En texto natural los tres algoritmos son lineales y Boyer-Moore es sublineal con patrones largos. En la entrada adversaria, Boyer-Moore y Rabin-Karp alcanzan exactamente su cota $O(n \cdot m)$ mientras KMP mantiene su garantía $O(n)$. La medición revela además las **constantes** que la notación O oculta: el costo de σ en el preprocesamiento de Boyer-Moore y el costo de las operaciones módulo de Rabin-Karp.

7. Aplicación Web

La interfaz web (`app/src/main/resources/static/`) es una aplicación de una sola página (HTML + CSS + JS vainilla, sin frameworks ni CDN). Tiene un encabezado con el título del proyecto y un menú de opciones; cada vista tiene su propia dirección (`#inicio`, `#similitud`, `#clustering`, …), así funcionan el botón "atrás" del navegador y la recarga de la página. La aplicación abre en **Inicio**.

Tabla 47

Opciones del Menú de la Aplicación Web

Opción del menú	Contenido
Inicio	Bienvenida al proyecto, frase motivacional, ilustración, cifras del proyecto (20 artículos, 3 requerimientos) y tarjetas de acceso a cada sección.
Integrantes	Foto en forma de burbuja y nombre completo de cada integrante del equipo.
Acerca del proyecto	Objetivo, resumen de cada requerimiento y metodología.
Corpus documental	Tabla de los 20 artículos con acceso al abstract completo.
Requerimientos	Submenú con tarjetas para R1, R2 y R3; cada una lleva a la solución del requerimiento, con un botón para volver y un selector R1 / R2 / R3.
Documentos	Visor y descarga de los documentos en PDF. Si el archivo no está en la instalación, la vista lo indica.

Nota. Elaboración propia.

Las soluciones de cada requerimiento muestran:

1. **Corpus:** tabla de los 20 artículos con acceso al abstract completo.
2. **Requerimiento 1:** selección dinámica de **dos o más** artículos mediante casillas (con opciones para seleccionar todos o limpiar), visualización de los abstracts elegidos y

comparación con los 6 algoritmos. Con dos artículos se muestra una tarjeta por algoritmo con el puntaje y el detalle del cálculo (distancia, puntaje de alineamiento, ángulo, términos con mayor peso, conjuntos e intersección, según el algoritmo). Con k > 2 se muestra además una **matriz de similitud k × k por algoritmo** (coloreada según el valor) y el detalle de cualquier par, que se elige en una lista o haciendo clic en la celda.

3. **Requerimiento 2:** las dos métricas internas (tabla de correlación cofenética de los 3 algoritmos, con el más coherente resaltado, y tabla de silueta con el tamaño de los grupos para los cortes k = 2..6), el dendrograma de cada criterio con la opción de cortarlo en 2 a 6 grupos (cada grupo con un color y la línea de corte dibujada), la lista de los grupos del corte y la lista de las 19 fusiones en orden, con su altura y los artículos que une.
4. **Requerimiento 3:** campo para escribir el patrón, opciones (distinguir mayúsculas, módulo de Rabin-Karp y abstract para la traza) y, como resultado: validación de que los 3 algoritmos coinciden, tabla de tiempos de preprocesamiento y de búsqueda con sus contadores (comparaciones, alineamientos, colisiones), las estructuras auxiliares del patrón (tabla pi, tablas de carácter malo y sufijo bueno, cálculo del hash por Horner), la traza paso a paso de cada algoritmo con el patrón dibujado sobre el texto en cada alineamiento, y las ocurrencias de cada artículo resaltadas en su contexto.

Todas las peticiones al servidor pasan por una función que, si la petición falla (por ejemplo, porque el servidor no responde), muestra un mensaje de error en la vista en lugar de perderlo en silencio. Además de la API, el servidor entrega los archivos de la interfaz: HTML, CSS, JavaScript, imágenes (PNG, JPG, SVG) y el PDF de la vista Documentos.

La API REST que consume el frontend:

Tabla 48

Rutas de la API REST

Endpoint	Descripción
`GET /api/articulos`	Lista los 20 artículos (metadatos + abstract).
`GET /api/articulos/{id}`	Un artículo específico.
`GET /api/similitud?ids=1,2,5`	Compara dos o más artículos: ejecuta los 6 algoritmos sobre cada uno de los k(k−1)/2 pares y devuelve los artículos, el resultado detallado de cada par y una matriz k × k por algoritmo. También acepta `?idA=1&idB=2` para un solo par.
`GET /api/clustering`	Devuelve, para los 3 criterios de enlace, el dendrograma, las 19 fusiones (altura y artículos de cada cluster), la correlación cofenética y los cortes k = 2..6 con sus grupos, tamaños y silueta.
`GET /api/busqueda?patron=&mayusculas=false&modulo=1000000007&traza=16&repeticiones=50`	Busca el patrón en los 20 abstracts con KMP, Boyer-Moore y Rabin-Karp; devuelve ocurrencias, tiempos, contadores, estructuras auxiliares y la traza del abstract indicado. Solo `patron` es obligatorio.

Nota. Elaboración propia.

8. Pruebas de Verificación

Se implementó una batería de pruebas manuales (`app/src/test/java/.../PruebasAlgoritmos.java`, sin JUnit por la misma razón declarada en la sección 2.3) que compara la salida de cada algoritmo contra casos de referencia matemáticamente conocidos: distancia de Levenshtein("kitten","sitting")=3, similitud de un texto consigo mismo = 1.0 en los 4 algoritmos aplicables, Jaccard = 0 para conjuntos disjuntos, Jaccard = 2/4 = 0.5 para un caso con palabras repetidas (con $|A \cup B| = 4$ por inclusión-exclusión), el Jaccard explícito comparado contra las operaciones de conjuntos de la librería estándar (usadas **solo** como oráculo en la prueba) en los 190 pares del corpus, la comparación de 4 artículos (6 pares, matrices simétricas con diagonal 1 y cada celda igual a la comparación individual de su par), y una prueba de clustering sobre 4 puntos sintéticos con estructura de 2 pares claramente separados, verificando que el algoritmo los fusiona en el orden correcto, que la correlación cofenética resultante es alta (>0.9), que el corte en 2 grupos recupera los dos pares y que la silueta es exactamente 9.5/10.5 (valor calculado a mano: a(i) = 1 y b(i) = 10.5 para los 4 puntos).

Sobre el corpus real, las 19 alturas de fusión de cada criterio (Single, Complete y Average) se comparan con un clustering ingenuo que, en cada paso, recalcula la distancia entre clusters recorriendo todos sus pares de artículos (mínimo, máximo o promedio) en vez de usar Lance-Williams; coinciden exactamente. También se verifica que cada corte k = 2..6 sea una partición de los 20 artículos y que los tamaños del corte en 6 grupos sean los de la sección 5.5.2.

Para la búsqueda de patrones (sección 6) se verifica: la función de prefijo de KMP para `ababaca` contra el valor publicado por Cormen et al. (2009): [0, 0, 1, 2, 3, 0, 1]; 5 casos pequeños con resultado conocido, incluidos patrones solapados (`aa` en `aaaaa` → 0, 1, 2, 3) y un patrón más largo que el texto; **3000 casos aleatorios** sobre alfabetos de 2 a 4 letras (texto y patrón muy repetitivos, que es donde fallaría una tabla de sufijo bueno o de fallos mal construida) comparados contra una búsqueda ingenua usada solo como oráculo en la prueba; que Rabin-Karp con q = 101 produzca colisiones espurias en el abstract 1 sin dejar de encontrar exactamente las ocurrencias correctas; y que los tres algoritmos coincidan sobre el corpus real para 5 patrones. Cada prueba se ejecuta para KMP, Boyer-Moore y Rabin-Karp (con q grande y con q = 101).

Para los dos algoritmos de IA se verifica que cada modelo cargue con sus dimensiones (100 en GloVe y 300 en FastText), que FastText cubra al menos 970 palabras del corpus, que cos(x, x) = 1 y la distancia euclidiana de un texto consigo mismo sea 0 en ambos enfoques, y que el coseno entre "teacher" y "student" en FastText coincida con un cálculo independiente hecho en la prueba.

9. Despliegue

Para mejor visualización del proceso de despliegue consulte la: Plantilla de despliegue

Referencias

Alfredo, R., Echeverria, V., Jin, Y., Yan, L., Swiecki, Z., Gašević, D. y Martinez-Maldonado, R. (2024). Human-centred learning analytics and AI in education: A systematic literature review. *Computers and Education: Artificial Intelligence*, *6*, Artículo 100215. https://doi.org/10.1016/j.caeai.2024.100215

Bahroun, Z., Anane, C., Ahmed, V. y Zacca, A. (2023). Transforming education: A comprehensive review of generative artificial intelligence in educational settings through bibliometric and content analysis. *Sustainability*, *15*(17), Artículo 12983. https://doi.org/10.3390/su151712983

Bewersdorff, A., Zhai, X., Roberts, J. y Nerdel, C. (2023). Myths, mis- and preconceptions of artificial intelligence: A review of the literature. *Computers and Education: Artificial Intelligence*, *4*, Artículo 100143. https://doi.org/10.1016/j.caeai.2023.100143

Bojanowski, P., Grave, E., Joulin, A. y Mikolov, T. (2017). Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics*, *5*, 135–146. https://doi.org/10.1162/tacl_a_00051

Bond, M., Khosravi, H., De Laat, M., Bergdahl, N., Negrea, V., Oxley, E., Pham, P., Chong, S. W. y Siemens, G. (2024). A meta systematic review of artificial intelligence in higher education: A call for increased ethics, collaboration, and rigour. *International Journal of Educational Technology in Higher Education*, *21*, Artículo 4. https://doi.org/10.1186/s41239-023-00436-z

Boyer, R. S. y Moore, J. S. (1977). A fast string searching algorithm. *Communications of the ACM*, *20*(10), 762–772. https://doi.org/10.1145/359842.359859

Chen, X., Xie, H. y Hwang, G.-J. (2020). A multi-perspective study on artificial intelligence in education: Grants, conferences, journals, software tools, institutions, and researchers. *Computers and Education: Artificial Intelligence*, *1*, Artículo 100005. https://doi.org/10.1016/j.caeai.2020.100005

Chen, X., Xie, H., Zou, D. y Hwang, G.-J. (2020). Application and theory gaps during the rise of artificial intelligence in education. *Computers and Education: Artificial Intelligence*, *1*, Artículo 100002. https://doi.org/10.1016/j.caeai.2020.100002

Chiu, T. K. F., Xia, Q., Zhou, X., Chai, C. S. y Cheng, M. (2023). Systematic literature review on opportunities, challenges, and future research recommendations of artificial intelligence in education. *Computers and Education: Artificial Intelligence*, *4*, Artículo 100118. https://doi.org/10.1016/j.caeai.2022.100118

Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2009). *Introduction to algorithms* (3.ª ed.). MIT Press.

Dai, C.-P. y Ke, F. (2022). Educational applications of artificial intelligence in simulation-based learning: A systematic mapping review. *Computers and Education: Artificial Intelligence*, *3*, Artículo 100087. https://doi.org/10.1016/j.caeai.2022.100087

Farris, J. S. (1969). On the cophenetic correlation coefficient. *Systematic Zoology*, *18*(3), 279–285. https://doi.org/10.2307/2412324

Gavhane, J. M. y Pagare, R. (2024). Artificial intelligence for education and its emphasis on assessment and adversity quotient: A review. *Education + Training*, *66*(6), 609–645. https://doi.org/10.1108/ET-04-2023-0117

Jaccard, P. (1912). The distribution of the flora in the alpine zone. *New Phytologist*, *11*(2), 37–50. https://doi.org/10.1111/j.1469-8137.1912.tb05611.x

Joksimovic, S., Ifenthaler, D., Marrone, R., De Laat, M. y Siemens, G. (2023). Opportunities of artificial intelligence for supporting complex problem-solving: Findings from a scoping review. *Computers and Education: Artificial Intelligence*, *4*, Artículo 100138. https://doi.org/10.1016/j.caeai.2023.100138

Karp, R. M. y Rabin, M. O. (1987). Efficient randomized pattern-matching algorithms. *IBM Journal of Research and Development*, *31*(2), 249–260. https://doi.org/10.1147/rd.312.0249

Kaufman, L. y Rousseeuw, P. J. (1990). *Finding groups in data: An introduction to cluster analysis*. Wiley. https://doi.org/10.1002/9780470316801

Knuth, D. E., Morris, J. H., Jr. y Pratt, V. R. (1977). Fast pattern matching in strings. *SIAM Journal on Computing*, *6*(2), 323–350. https://doi.org/10.1137/0206024

Lance, G. N. y Williams, W. T. (1967). A general theory of classificatory sorting strategies: 1. Hierarchical systems. *The Computer Journal*, *9*(4), 373–380. https://doi.org/10.1093/comjnl/9.4.373

Laupichler, M. C., Aster, A., Schirch, J. y Raupach, T. (2022). Artificial intelligence literacy in higher and adult education: A scoping literature review. *Computers and Education: Artificial Intelligence*, *3*, Artículo 100101. https://doi.org/10.1016/j.caeai.2022.100101

Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, *10*(8), 707–710.

Lo, C. K. (2023). What is the impact of ChatGPT on education? A rapid review of the literature. *Education Sciences*, *13*(4), Artículo 410. https://doi.org/10.3390/educsci13040410

Lye, C. Y. y Lim, L. (2024). Generative artificial intelligence in tertiary education: Assessment redesign principles and considerations. *Education Sciences*, *14*(6), Artículo 569. https://doi.org/10.3390/educsci14060569

Mikeladze, T., Meijer, P. C. y Verhoeff, R. P. (2024). A comprehensive exploration of artificial intelligence competence frameworks for educators: A critical review. *European Journal of Education*, *59*, Artículo e12663. https://doi.org/10.1111/ejed.12663

Mikolov, T., Grave, E., Bojanowski, P., Puhrsch, C. y Joulin, A. (2018). Advances in pre-training distributed word representations. En *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018)* (pp. 52–55). European Language Resources Association.

Needleman, S. B. y Wunsch, C. D. (1970). A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology*, *48*(3), 443–453. https://doi.org/10.1016/0022-2836(70)90057-4

Ogunleye, B., Zakariyyah, K. I., Ajao, O., Olayinka, O. y Sharma, H. (2024). A systematic review of generative AI for teaching and learning practice. *Education Sciences*, *14*(6), Artículo 636. https://doi.org/10.3390/educsci14060636

Ouyang, F., Dinh, T. A. y Xu, W. (2023). A systematic review of AI-driven educational assessment in STEM education. *Journal for STEM Education Research*, *6*, 408–426. https://doi.org/10.1007/s41979-023-00112-x

Pennington, J., Socher, R. y Manning, C. D. (2014). GloVe: Global vectors for word representation. En *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 1532–1543). Association for Computational Linguistics. https://doi.org/10.3115/v1/D14-1162

Porter, M. F. (1980). An algorithm for suffix stripping. *Program*, *14*(3), 130–137. https://doi.org/10.1108/eb046814

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, *20*, 53–65. https://doi.org/10.1016/0377-0427(87)90125-7

Salton, G. y McGill, M. J. (1983). *Introduction to modern information retrieval*. McGraw-Hill.

Sokal, R. R. y Rohlf, F. J. (1962). The comparison of dendrograms by objective methods. *Taxon*, *11*(2), 33–40. https://doi.org/10.2307/1217208

Tan, S. C., Lee, A. V. Y. y Lee, M. (2022). A systematic review of artificial intelligence techniques for collaborative learning over the past two decades. *Computers and*

Education: Artificial Intelligence, *3*, Artículo 100097. https://doi.org/10.1016/j.caeai.2022.100097

Wagner, R. A. y Fischer, M. J. (1974). The string-to-string correction problem. *Journal of the ACM*, *21*(1), 168–173. https://doi.org/10.1145/321796.321811

Xia, Q., Weng, X., Ouyang, F., Lin, T. J. y Chiu, T. K. F. (2024). A scoping review on how generative artificial intelligence transforms assessment in higher education. *International Journal of Educational Technology in Higher Education*, *21*, Artículo 40. https://doi.org/10.1186/s41239-024-00468-z

Zawacki-Richter, O., Marín, V. I., Bond, M. y Gouverneur, F. (2019). Systematic review of research on artificial intelligence applications in higher education – where are the educators? *International Journal of Educational Technology in Higher Education*, *16*, Artículo 39. https://doi.org/10.1186/s41239-019-0171-0

Zhang, K. y Aslan, A. B. (2021). AI technologies for education: Recent research & future directions. *Computers and Education: Artificial Intelligence*, *2*, Artículo 100025. https://doi.org/10.1016/j.caeai.2021.100025

Anexos:

Video 1 Algoritmo de Levenshtein

Video 2 Algoritmo Levenshtein

Video 3 Algoritmo de Needleman Wunsch

Video 4 Algoritmo Needleman Wunsch

Video 6 Algoritmo de Jaccard

Video 7 Algoritmos de IA

Video 8 construcción de la matriz de distancias

Video 9 Analisis matemático y ejecución de aglomeramiento

Video 10 patron de busqueda del algoritmo Knuth-Morris-Pratt

Video 11 llenado de la función de prefijo

Plantilla de despliegue

VIGILADA MINEDUCACIÓN

